

Tong Xiao

Jingbo Zhu

Natural Language Processing

Neural Networks and Large Language Models

NATURAL LANGUAGE PROCESSING LAB

NORTHEASTERN UNIVERSITY

&

NIUTRANS RESEARCH

October 7, 2026

Copyright © 2021-2026 Tong Xiao and Jingbo Zhu

NATURAL LANGUAGE PROCESSING LAB, NORTHEASTERN UNIVERSITY
&
NIUTRANS RESEARCH

<https://github.com/NiuTrans/NLPBook>

<https://niutrans.github.io/NLPBook>

Licensed under the Creative Commons Attribution-NonCommercial 4.0 Unported License (the “License”). You may not use this file except in compliance with the License. You may obtain a copy of the License at <http://creativecommons.org/licenses/by-nc/4.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

October 7, 2026

Chapter 4

Recurrent and Convolutional Sequence Models

The whole is more than the sum of its parts.

– Aristotle, 384-322 BC [Ross, 1924]

Aristotle might or might not have had linguistic phenomena in mind when he expressed this thought, but it is indeed what we want to convey in this chapter: a sentence or phrase is much more than a mere collection of words. While words carry meaning on their own, when they combine to form a sentence or phrase, the meaning of the whole can be significantly more complex and diverse. This leads to the most beautiful aspect of language: human beings can express an infinite variety of meanings using a finite set of elements (e.g., words or characters).

The infinite and highly context-dependent nature of language makes modeling a sequence of words far more difficult than modeling individual words. One challenge is that a word’s sense can change drastically depending on its context. Building upon the concept of word embeddings, where a word is represented as a low-dimensional, real-valued vector, the “meaning” of a language unit can be modeled in a continuous space. It is therefore possible to extend methods of distributed representation from single words to entire sequences. This motivates the exploration of models that process variable-length word sequences in a fundamentally continuous manner.

Here, we explore the general approach to learning distributed representations of word sequences. Specifically, we consider recurrent and convolutional neural networks, which have been extensively used in fields ranging from speech processing to computer vision. For natural language inputs, applying these models yields a sequence-level representation. This representation can be either a single real-valued vector or a sequence of such vectors, each corresponding to a contextualized representation for a specific word within the input sequence. These representation models, often referred to as encoders, are commonly used in a variety of systems that process sequential data. We will see several examples of these in this chapter.

4.1 Problem Statement

For many NLP applications, our objective is to make predictions based on an input sequence. Let us consider again the text classification problem mentioned in Chapter 1. Given a text that may or may not discuss food, we want to assign one of two classes to it (e.g., Food or Not-food). To do this, a common classification method is to represent the text as a bag of features, denoted as $\mathbf{H}$. Then, a probability is assigned to each of the classes using a probabilistic model $\Pr(y|\mathbf{H})$. The predicted class is the one that maximizes this probability $\hat{y} = \arg \max_y \Pr(y|\mathbf{H})$.

While this is a standard procedure for classification, the underlying concept describes a much broader problem. Formally, let $\mathbf{w} = w_1 \cdots w_m$ be a sequence of words¹. A sequence-level NLP system can be formulated as a function that maps the input sequence $\mathbf{w}$ to some output $\mathbf{y}$. This process can be divided into two distinct steps: representation (or encoding) and prediction.

- **Representation (or Encoding).** This step transforms the input sequence $\mathbf{w}$ to a feature representation $\mathbf{H}$ using an encoder $\text{Encode}(\cdot)$:

$$\mathbf{H} = \text{Encode}(\mathbf{w}). \quad (4.1)$$

- **Prediction.** A predictor $\text{Predict}(\cdot)$ takes $\mathbf{H}$ and generates the output:

$$\mathbf{y} = \text{Predict}(\mathbf{H}). \quad (4.2)$$

A simple form of $\mathbf{H}$ is a feature vector. For example, $\mathbf{H}$ could be a set of human-designed indicator features extracted from $\mathbf{w}$ (representing a high-dimensional sparse representation), or a set of real numbers indicating latent features (representing a low-dimensional dense representation). In NLP, another common form of $\mathbf{H}$ is a sequence of vectors, where each vector $\mathbf{h}_i$ corresponds to an input word w_i (see Figure 4.1). In this case, $\mathbf{h}_i$ can be viewed as a contextualized representation of both w_i and its surrounding context in $\mathbf{w}$ ². The correspondence between $\mathbf{h}_i$ and w_i enables the representation to distinguish between different positions in the sequence and, more importantly, to dynamically adapt its modeling power for variable-length inputs.

The form of $\mathbf{y}$ depends on the problem we intend to deal with. For example, in classification problems, $\mathbf{y}$ is the index of a class (or a distribution of classes); for regression problems, $\mathbf{y}$ is a real number; and in machine translation, $\mathbf{y}$ is a sequence of words in the target language. Note that, in this model, representation and prediction can be regarded as two separate problems. A major advantage of decoupling them is that the same encoder can be reused across multiple applications with different predictors. This also motivates a highly promising line of research where a general-purpose encoder is pre-trained on large-scale data and then used as a core

¹Although we restrict our discussion to word sequences, these methods can handle sequences of any language units, such as sub-words or characters.

²This architecture can be extended to encoders where the input and output have different lengths. For example, the input is $w_1 \cdots w_m$ and the output is $\mathbf{h}_1 \cdots \mathbf{h}_n$ ($m \neq n$).

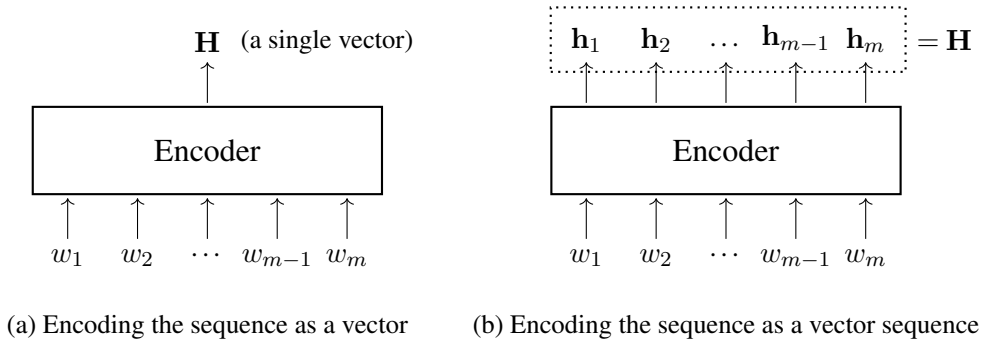


Figure 4.1: Representing a word sequence as (a) a vector or (b) a sequence of vectors.

component in various downstream systems [Peters et al., 2018; Devlin et al., 2019].

There are many possible implementations for $\text{Encode}(\cdot)$ and $\text{Predict}(\cdot)$. For example, in text classification, one way is to define $\text{Encode}(\cdot)$ as a function that computes a feature vector using hand-crafted templates, while defining $\text{Predict}(\cdot)$ as a traditional statistical classifier (e.g., an SVM or a maximum entropy model). Another way is to define $\text{Encode}(\cdot)$ as a multi-layer neural network that outputs a real-valued vector, and define $\text{Predict}(\cdot)$ as a simple neural network consisting of a single Softmax layer. In this chapter, we focus on neural network-based encoders. Section 4.5 will demonstrate how these encoders can be applied to a variety of NLP tasks.

4.2 Recurrent Models

A comprehensive study of various sequence models is not easy work. It is convenient, however, to first introduce one of the most common and practical neural models, called recurrent neural networks (RNNs). We will see later that RNNs are extensively used in sequence modeling, and the techniques presented here are generic and applicable to many systems.

4.2.1 RNN-based Language Models

Perhaps the most prevalent application of sequence models in NLP is estimating the probability of a word sequence, known as language modeling. Mathematically, language modeling is a specific instance of a fundamental problem in the field of **stochastic processes** (or **random processes**): the modeling of **time series** data [Hamilton, 1994; Chatfield, 2003; Fuller, 2009]. Treated as a time series, a sequence of words constitutes discrete data points occurring at sequential time steps. In this sense, the methods presented here are broadly applicable, although an exhaustive discussion on general time series analysis falls outside the scope of this text.

Given a sequence of words $w_1 \dots w_m$, the goal of language modeling is to compute $\Pr(w_1, \dots, w_m)$. This joint probability is typically written as a product of conditional probabilities using the chain rule:

$$\Pr(w_1, \dots, w_m) = \Pr(w_1) \cdot \Pr(w_2|w_1) \cdots \Pr(w_m|w_1, \dots, w_{m-1}). \quad (4.3)$$

In other words, the problem of generating $w_1 \cdots w_m$ is the same as the problem of generating a word w_{i+1} at a time based on the previous words $w_1 \cdots w_i$. RNN-based language models represent $w_1 \cdots w_i$ via a recurrent unit $\text{RNN}(\cdot)$ [Mikolov et al., 2010]:

$$\mathbf{h}_i = \text{RNN}(\mathbf{h}_{i-1}, \mathbf{x}_i), \quad (4.4)$$

where $\mathbf{x}_i \in \mathbb{R}^{d_e}$ is the word vector (or word embedding) for w_i . Let V be the vocabulary from which we can choose a word. If $\mathbf{1}_{w_i}$ is the one-hot representation of w_i , then $\mathbf{x}_i$ is given by multiplying $\mathbf{1}_{w_i}$ with the word embedding table $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$:

$$\begin{aligned} \mathbf{x}_i &= \text{Embed}(w_i) \\ &= \mathbf{1}_{w_i} \mathbf{C}. \end{aligned} \quad (4.5)$$

As shown in Chapter 1, the use of $\mathbf{C}$ transforms a $|V|$ -dimensional (and probably high-dimensional) vector to a d_e -dimensional (and probably low-dimensional) vector. Note that $\mathbf{C}$ is essentially a lookup table, with a distinct table entry (i.e., a row) for each word in V . So, the right-hand side of Eq. (4.5) is a lookup operation which extracts the corresponding row from $\mathbf{C}$.

Now we go back to Eq. (4.4). Intuitively, this equation implies that the state of the context we have seen so far (i.e., $\mathbf{h}_i$) is some representation of the combination of the current input (i.e., $\mathbf{x}_i$) and the state of the earlier context ($\mathbf{h}_{i-1}$). Put another way, it can be thought of as a process of repeatedly adding information of a new word to a cache of “history”. An elegant aspect of this process is that it can be easily implemented by repeatedly applying Eq. (4.4) until the end of the sequence.

$\text{RNN}(\cdot)$ can be any function that takes $\mathbf{h}_{i-1}$ and $\mathbf{x}_i$, and produces a new vector $\mathbf{h}_i$. The vanilla RNN has a form

$$\text{RNN}(\mathbf{h}_{i-1}, \mathbf{x}_i) = \psi(\mathbf{h}_{i-1} \mathbf{U} + \mathbf{x}_i \mathbf{V}), \quad (4.6)$$

where $\psi(\cdot)$ is an activation function, such as $\text{TanH}(\cdot)$ and $\text{Sigmoid}(\cdot)$. Together with Eqs. (4.4) and (4.5), we can define $\mathbf{h}_i$ as a function of $\mathbf{h}_{i-1}$ and w_i

$$\mathbf{h}_i = \psi(\mathbf{h}_{i-1} \mathbf{U} + \mathbf{1}_{w_i} \mathbf{C} \mathbf{V}), \quad (4.7)$$

where $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V} \in \mathbb{R}^{d_e \times d_h}$, and $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ are learnable parameters of the model, and d_h is a hyper-parameter indicating the number of dimensions of $\mathbf{h}_i$ and $\mathbf{h}_{i-1}$.

We now have an encoder that represents the word sequence $w_1 \cdots w_m$ as a sequence of RNN’s outputs $\mathbf{H} = \{\mathbf{h}_1, \dots, \mathbf{h}_m\}$. Given that each $\mathbf{h}_i$ encodes the sub-sequence spanning from w_1 to w_i , we can place a Softmax layer on $\mathbf{h}_i$ to obtain a distribution of words:

$$\mathbf{y}_{i+1} = \text{Softmax}(\mathbf{h}_i \mathbf{O} + \mathbf{b}), \quad (4.8)$$

where $\mathbf{O} \in \mathbb{R}^{d_h \times |V|}$ and $\mathbf{b} \in \mathbb{R}^{|V|}$. Taking the word index w_{i+1} , we have

$$\Pr(w_{i+1}|w_1, \dots, w_i) = y_{i+1}(w_{i+1}). \quad (4.9)$$

Thus, we have developed a language model that produces a probability $\Pr(w_{i+1}|w_1, \dots, w_i)$ at each step. Figure 4.2 shows an illustration of the RNN-based language model for an example sequence. To run this model on a word sequence, we surely wish to start with predicting w_1 but this requires a preceding word w_0 that is taken as the input. A simple and widely applicable method for giving an appropriate starting state to RNNs is to add a beginning symbol $\langle \text{SOS} \rangle$ to the sequence so that all sequences start with the same “word”. Likewise, we can attach an end symbol $\langle \text{EOS} \rangle$ to the sequence to model the completeness of the sequence. This leads to a new form of the probability of the sequence

$$\begin{aligned} \Pr(\langle \text{SOS} \rangle, w_1, \dots, w_m, \langle \text{EOS} \rangle) &= \Pr(\langle \text{SOS} \rangle) \cdot \\ &\quad \Pr(w_1 | \langle \text{SOS} \rangle) \cdot \\ &\quad \Pr(w_2 | \langle \text{SOS} \rangle, w_1) \cdot \\ &\quad \dots \\ &\quad \Pr(w_m | \langle \text{SOS} \rangle, w_1, \dots, w_{m-1}) \cdot \\ &\quad \Pr(\langle \text{EOS} \rangle | \langle \text{SOS} \rangle, w_1, \dots, w_m). \end{aligned} \quad (4.10)$$

We can simply assume $\Pr(\langle \text{SOS} \rangle) = 1$. To obtain $\Pr(\langle \text{SOS} \rangle, w_1, \dots, w_m, \langle \text{EOS} \rangle)$, we take $\langle \text{SOS} \rangle w_1 \dots w_m$ as an input sequence and $w_1 \dots w_m \langle \text{EOS} \rangle$ as the output sequence.

4.2.2 Training

As a differentiable neural architecture, the RNN-based language model can be trained using standard optimization techniques. The foundational principles of training neural networks have been thoroughly discussed in Chapter 2. Therefore, rather than providing an exhaustive description here, we outline the fundamental concepts and discuss several refinements to sequence models.

RNN-based language modeling can be framed as a next-step-prediction problem. Suppose we are given a corpus $\mathcal{D}$ consisting of word sequences. For each sequence $\mathbf{w} = w_1, \dots, w_{|\mathbf{w}|}$ in $\mathcal{D}$, we construct a sequence of input-target pairs, structured as follows³:

$$\{(w_1, w_2), (w_2, w_3), \dots, (w_{|\mathbf{w}|-1}, w_{|\mathbf{w}|})\}.$$

The language model processes the input prefix sequence $w_1, \dots, w_{|\mathbf{w}|-1}$ and emits a corresponding sequence of predicted probability distributions $\hat{\mathbf{y}}_2, \dots, \hat{\mathbf{y}}_{|\mathbf{w}|}$. See the following table for an illustration of the inputs and outputs of the model.

³While the $\langle \text{SOS} \rangle$ and $\langle \text{EOS} \rangle$ boundary tokens are essential in real-world implementations, we omit them from the notation here for simplicity.

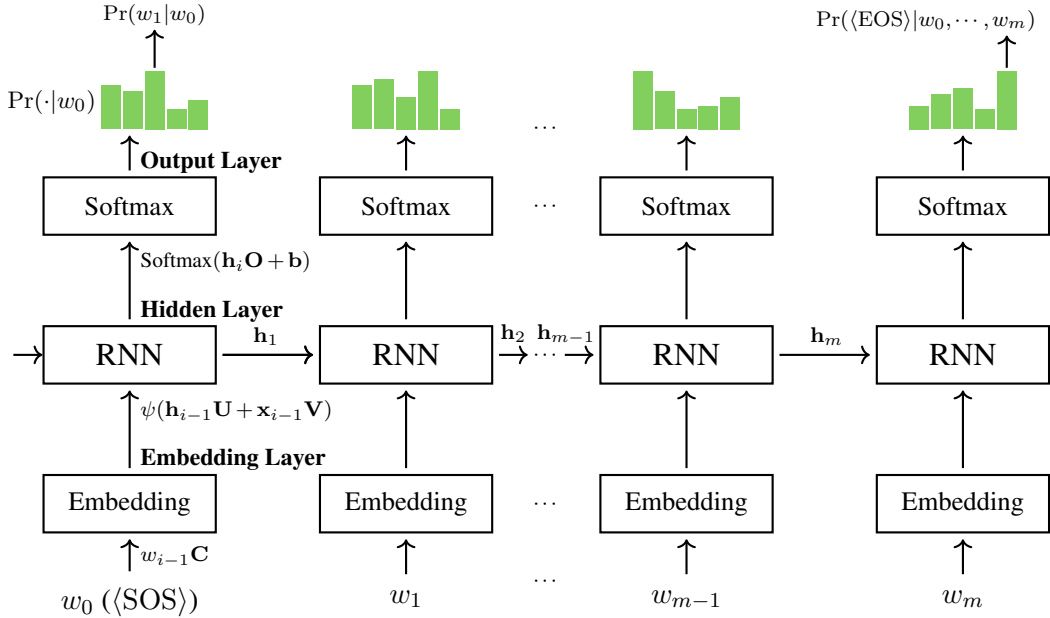


Figure 4.2: Illustration of using an RNN-based language model to calculate $\Pr(\langle \text{SOS} \rangle w_1 \cdots w_m \langle \text{EOS} \rangle)$. The input is $\langle \text{SOS} \rangle w_1 \cdots w_m$, and the output is the probability $\Pr(w_1 | \langle \text{SOS} \rangle) \Pr(w_2 | \langle \text{SOS} \rangle w_1) \cdots \Pr(\langle \text{EOS} \rangle | \langle \text{SOS} \rangle w_1 \cdots w_m)$. As $\Pr(\langle \text{SOS} \rangle) = 1$, the probability of generating the sequence is simply $\Pr(\langle \text{SOS} \rangle w_1 \cdots w_m \langle \text{EOS} \rangle) = \Pr(\langle \text{SOS} \rangle) \Pr(w_1 | \langle \text{SOS} \rangle) \Pr(w_2 | \langle \text{SOS} \rangle w_1) \cdots \Pr(\langle \text{EOS} \rangle | \langle \text{SOS} \rangle w_1 \cdots w_m)$. For each input w_i , we first represent it as a word vector $\mathbf{x}_i$ via the embedding layer, resulting in a sequence of word vectors $\mathbf{x}_0 \cdots \mathbf{x}_m$. The RNN layer maps $\mathbf{x}_0 \cdots \mathbf{x}_m$ to a sequence of hidden states $\mathbf{h}_1 \cdots \mathbf{h}_{m+1}$. In this process, the operation is applied recursively: an RNN unit takes both $\mathbf{h}_{i-1}$ and $\mathbf{x}_{i-1}$ and produces a new state $\mathbf{h}_i$. On top of that, we use the output layer (Softmax) to obtain $\Pr(w_{i+1} | \langle \text{SOS} \rangle w_1 \cdots w_i)$.

Step	History	Input	Output (Distribution)	Gold- Standard
1		w_1	$\hat{\mathbf{y}}_2$	w_2
2	w_1	w_2	$\hat{\mathbf{y}}_3$	w_3
3	w_1, w_2	w_3	$\hat{\mathbf{y}}_4$	w_4
...	...	...	...	...
$ \mathbf{w} - 2$	$w_1, w_2, \dots, w_{ \mathbf{w} -3}$	$w_{ \mathbf{w} -2}$	$\hat{\mathbf{y}}_{ \mathbf{w} -1}$	$w_{ \mathbf{w} -1}$
$ \mathbf{w} - 1$	$w_1, w_2, \dots, w_{ \mathbf{w} -3}, w_{ \mathbf{w} -2}$	$w_{ \mathbf{w} -1}$	$\hat{\mathbf{y}}_{ \mathbf{w} }$	$w_{ \mathbf{w} }$

A loss function $\mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i})$ is defined to quantify the discrepancy between the predicted distribution $\hat{\mathbf{y}}_i$ and the true one-hot target representation $\mathbf{1}_{w_i}$. The standard choice for this task is the cross-entropy loss, which measures the divergence between the two distributions [Mitchell, 1997; Bishop, 2006].

Then, the loss over the entire set is defined to be:

$$\mathcal{L} = \sum_{\mathbf{w} \in \mathcal{D}} \sum_{i=2}^{|\mathbf{w}|} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{1}_{w_i}). \quad (4.11)$$

Once we know the loss, the training of the RNN-based language model can be achieved by using gradient descent. A simple form of this method is the delta rule

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}, \quad (4.12)$$

where θ stands for the parameters. For the model described in Section 4.2.1, θ includes $\mathbf{C}$, $\mathbf{U}$, $\mathbf{V}$, $\mathbf{O}$ and $\mathbf{b}$. $\frac{\partial \mathcal{L}}{\partial \theta}$ is the derivative of the loss with respect to the parameters, called **error gradient**.

Eq. (4.12) can be understood as a process of moving the current parameters a small step in the steepest downhill direction (i.e., the direction of $-\frac{\partial \mathcal{L}}{\partial \theta}$). Here η stands for how far we move in each step of going downhill, also called the learning rate. Obtaining $\frac{\partial \mathcal{L}}{\partial \theta}$ often requires a back-propagation algorithm that propagates errors backward through the unrolled time steps. In modern implementations of deep learning systems, in which neural networks are represented as computation graphs, back-propagation is simple since it is just a by-product of graph traversal and there are many automatic differentiation toolkits to do this. Similar algorithms, called **back-propagation through time (BPTT)**, were also used in earlier systems [Werbos, 1990]. For further information about training neural networks, see Chapter 2 and/or textbooks on this subject [Goodfellow et al., 2016; Zhang et al., 2021].

If the input is a long sequence, the application of RNNs would result in a deep neural network. In this case, the use of the chain rule of ordered derivatives makes large or small loss derivatives accumulate, and the update to the parameters in Eq. (4.12) is consequently very large or small. These are typically known as the **exploding and vanishing gradient problems**. There are several methods to mitigate these problems for RNNs [Sutskever, 2013]. Some of them are

- **Regularization.** Introducing regularization terms (such as the l_1 and l_2 norms on parameter matrices) into training can avoid models in which most of the parameters have large values, and thus help to avoid exploding gradients. Similarly, one can penalize the cases in which the norms of the gradients are too small [Pascanu et al., 2013].
- **Gradient Clipping.** When the norm of the gradients is too large, it is natural to directly scale down their magnitudes. A simple method is to clip the gradient norm in terms of a threshold τ . If the norm $\|\frac{\partial \mathcal{L}}{\partial \theta}\|$ is larger than τ , we can rescale $\frac{\partial \mathcal{L}}{\partial \theta}$ accordingly, e.g., $\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\tau}{\|\frac{\partial \mathcal{L}}{\partial \theta}\|} \cdot \frac{\partial \mathcal{L}}{\partial \theta}$.⁴

⁴It is usually formulated as an equation

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\tau}{\max(\tau, \|\frac{\partial \mathcal{L}}{\partial \theta}\|)} \cdot \frac{\partial \mathcal{L}}{\partial \theta}. \quad (4.13)$$

- **Truncated Back-propagation.** Another idea is to break a long sequence of input-output pairs into shorter pieces, and train RNNs on these separate sub-sequences [Williams and Peng, 1990; Elman, 1990]. This reduces both the cost of training and the risk of too large or small values in accumulating error gradients.
- **Improved Architectures.** It is also possible to redesign the model to overcome the limits of standard RNNs, usually using the memory mechanism. In Section 4.3, we will see a few examples of redesigning the recurrent unit for addressing the vanishing gradient problem.
- **Initialization and Constraints of Parameters.** Initializing the model parameters to a desirable region is generally helpful for optimization, and, sometimes, helpful for preventing very small gradients. An alternative method is to randomly set the model and only learn the parameters of the output layers [Jaeger and Haas, 2004].
- **Non-saturating Activations.** Many common activation functions have a compact range of outputs, e.g., the Sigmoid function has a range of $[0, 1]$. They are also called **saturating activation functions**⁵. The use of saturating activation functions often leads to the decay of gradients over layers, i.e., the vanishing gradient. It is therefore promising to use non-saturating activation functions instead, e.g., the ReLU function.
- **Normalization of Activations.** Saturating activations may also result in getting stuck in a saturated region of outputs, and we need a large learning rate to escape from local optima [Ioffe and Szegedy, 2015]. Thus, the training would be unstable, and subtle changes in inputs and/or model parameters would lead to a big variance in model behavior. A possible solution is to normalize the activations to reduce the variance, e.g., subtracting the mean of the activations in a group of samples (e.g., samples in a mini-batch of training), and dividing by their standard deviation.

4.2.3 Layer Stacking

If we think of the application of a recurrent unit as a function mapping a variable sequence to a new variable sequence of the same length, it is natural to compose this function with another function of the same type, or even with itself. This makes it very easy to extend RNNs to deep neural networks simply by stacking recurrent layers.

Let $\mathbf{h}_i^l$ be the output of the l -th recurrent unit in the stack at position i . We can apply a new recurrent unit to $\mathbf{h}_i^l$, resulting in a new output at level $l + 1$

$$\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l), \quad (4.14)$$

where $\mathbf{h}_{i-1}^{l+1}$ is the output of the previous step at level $l + 1$. To make Eq. (4.14) well-formed, we typically define $\mathbf{h}_i^0 = \mathbf{x}_i$. In other words, the stack starts off with the word vector $\mathbf{x}_i$, then a series of RNN outputs (i.e., $\mathbf{h}_i^1, \mathbf{h}_i^2, \mathbf{h}_i^3$, etc).

To illustrate, Figure 4.3 (a) shows a stacked RNN for language modeling. We see that

⁵An activation function $f(x)$ is non-saturating if and only if when $x \rightarrow \infty$ (or $-\infty$), $f(x) \rightarrow \infty$. An activation function is saturating if it is not a non-saturating activation function.

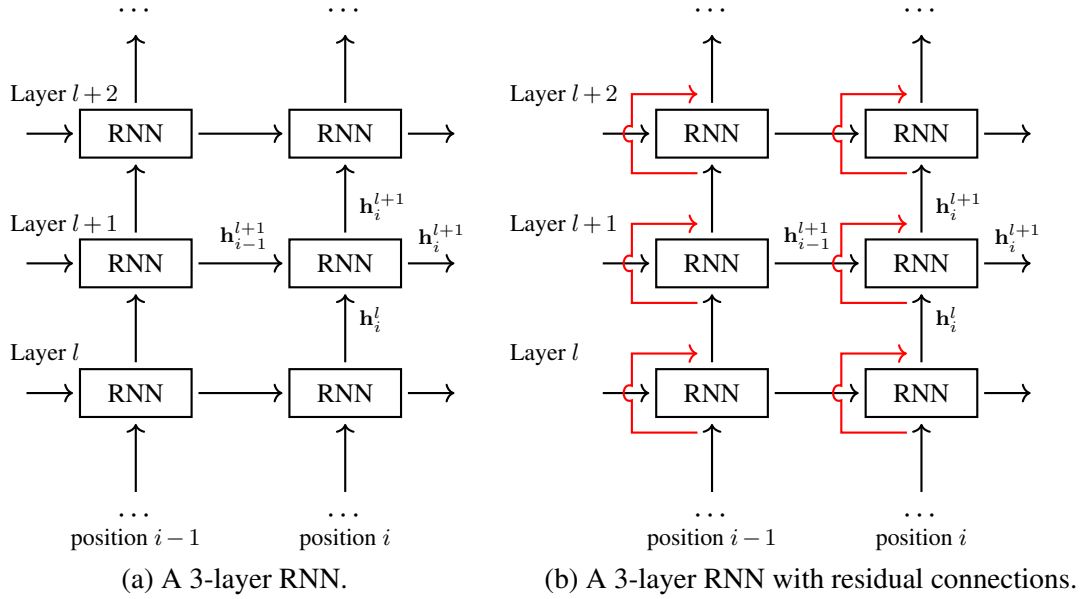


Figure 4.3: 3-layer RNNs (with and without residual connections). To stack RNN layers, we feed the output of layer l to layer $l+1$. Thus the output of layer $l+1$ is given by $\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l)$. Red lines stand for the residual connections which directly add the input of a layer to its output, resulting in $\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l) + \mathbf{h}_i^l$.

applying a stack of recurrent units is equivalent to creating multiple layers of RNNs simultaneously. However, there would be a risk of confusion if we call an unrolled recurrent network a *layer*, as the term *layer* typically refers to a set of neurons receiving the same inputs in a feed-forward neural network. Here we extend the term *layer* to cover a more general concept: a group of neurons that are topologically placed on the same level. So, we say that the language model in Figure 4.3 has 3 RNN layers.

Stacking multiple layers of RNNs, we build a model which is deeper but more difficult to train. This difficulty arises in part from the barriers of passing information through many-layered RNNs. To make the training easier, a widely-used approach is to introduce **skip connections** or **residual connections** into a multi-layer neural network [He et al., 2016]. These connections are intended to leverage an additional path to allow information to skip layers. As described in Chapter 2, the form of a residual neural network is given by:

$$\mathbf{y}^{l+1} = F(\mathbf{y}^l) + \mathbf{y}^l, \quad (4.15)$$

where $\mathbf{y}^l$ is the output of layer l . Extending this formulation to Eq. (4.14) leads to multi-layer RNNs with residual connections, given by:

$$\mathbf{h}_i^{l+1} = \text{RNN}(\mathbf{h}_{i-1}^{l+1}, \mathbf{h}_i^l) + \mathbf{h}_i^l. \quad (4.16)$$

The only difference from Eq. (4.14) is that we introduce the identity map of $\mathbf{h}_i^l$ to the

right-hand side of Eq. (4.16). Thus, the input $\mathbf{h}_i^l$ is directly accessible from layer $l + 1$. This greatly simplifies the way that the information flows through the neural network, and allows the system to “skip” layers in propagating errors. Figure 4.3 (b) shows a 3-layer RNN with residual connections.

4.2.4 Bi-directional Models

The use of RNNs enables us to formulate the problem of encoding a word sequence as a problem of left-to-right generation of words. One advantage of this approach is that the modeling of contextual information arises naturally: the output of an RNN unit in some way describes the context words up to that point. This feature makes it very straightforward to model the probability distribution $\Pr(w_{i+1}|w_1, \dots, w_i)$, as we can use $\mathbf{h}_i$ as a representation of the context $w_1 \cdots w_i$, that is, $\Pr(w_{i+1}|w_1, \dots, w_i) = \Pr(w_{i+1}|\mathbf{h}_i)$.

The left-to-right generation is widely used in sequence generation, such as machine translation. It can be viewed as an instance of **autoregressive processes (AR processes)** in which the state of a variable is dependent on the state of the previous variables [Chatfield, 2003; Box et al., 2015]⁶. However, such a method is not the only choice for modeling sequences. We do not even necessarily restrict ourselves to language modeling for training a sequence encoder. This gives rise to an interesting question: how can we develop an encoder of word sequences without assumptions regarding the predictor? Answering the question leads us to isolate the learning of the text encoder from a specific NLP task, and to regard it as a separate task whose result can be applied to many other systems. A more detailed discussion is not the focus here and we leave it to subsequent chapters.

We now present a simple extension of the left-to-right sequence model by returning to RNNs. Note that in sequence modeling our desire is some representation of the entire sequence. A problem with usual RNNs is that they are **uni-directional models** in which the context words following w_i are absent. To consider both the left and right contexts of a given word, we can instead use **bi-directional models**. Figure 4.4 shows an example of the bi-directional RNN. There are two sub-models: a left-to-right RNN and a right-to-left RNN. They have the same architecture but work in opposite directions. For each input word w_i , the left-to-right

⁶As a stochastic process, an autoregressive process expresses a variable at time t by relating it to the past values of the process and the current value of an error process [Chatfield, 2003]. Formally, a time series $\{z_1, \dots, z_T\}$ describes an autoregressive process of order p if for any $t \in \{p + 1, \dots, T\}$

$$z_t = \sum_{i=1}^p \alpha_i z_{t-i} + \epsilon_t, \quad (4.17)$$

where $\{\alpha_1, \dots, \alpha_p\}$ are the parameters of the process, and ϵ_t is the error at time t . This process is called *regressive* because it has the same form as the **multiple linear regression model**. The prefix *auto-* comes from the way we regress z_t : z_t is dependent on its past values instead of additional independent variables. One way to interpret language modeling in an autoregressive process perspective is to simply treat $\{z_1, \dots, z_T\}$ as representations of a sequence of words $\{w_1, \dots, w_T\}$. Thus, we can gain some idea of predicting w_t using previous words $\{w_1, \dots, w_{t-1}\}$ by considering the autoregressive property of the problem. However, it should be noted that most of the sequence generation models used in NLP are not mathematically equivalent to Eq. (4.17), although they are often called regressive models. For example, the RNN-based language model discussed here is not a linear model. Rather, it takes layers of non-linearity to describe the complex relationships among words.

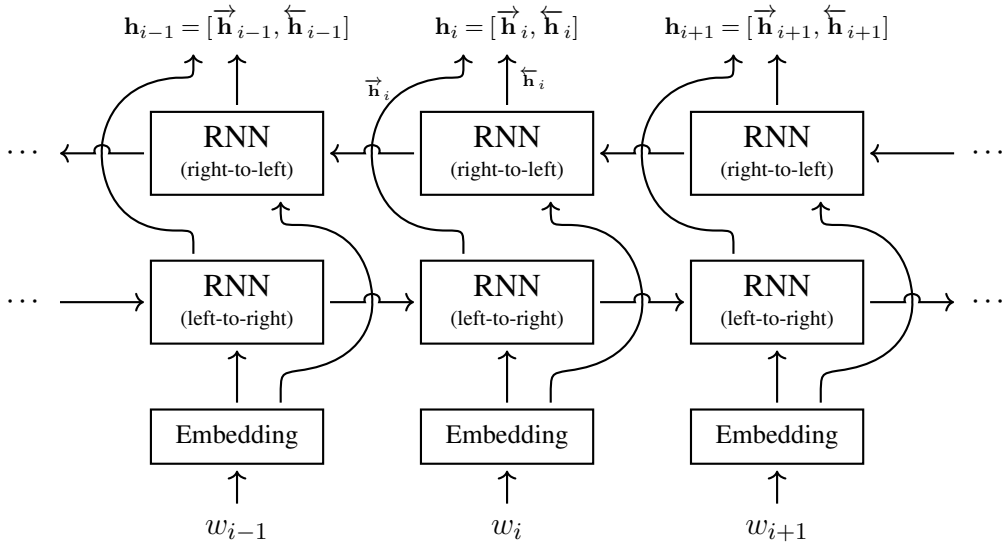


Figure 4.4: A bi-directional RNN model. Given a word sequence, we run an RNN from left to right and another RNN from right to left. Therefore, at each position we obtain a left-to-right representation and a right-to-left representation. The output is the concatenation of the two representations so that it involves both the left and right contexts.

RNN outputs a vector representing the context $\{w_1, \dots, w_i\}$ (denoted by $\vec{h}_i$), and the right-to-left RNN outputs a vector representing the context $\{w_i, \dots, w_m\}$ (denoted by $\overleftarrow{h}_i$). We can concatenate $\vec{h}_i$ and $\overleftarrow{h}_i$ to obtain a bi-directional representation

$$\mathbf{h}_i = [\vec{h}_i, \overleftarrow{h}_i]. \quad (4.18)$$

Thus, the bi-directional RNN has the same form of output as that of the uni-directional RNN, that is, a sequence of vectors $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$. Unlike the uni-directional RNN, the representation $\mathbf{h}_i$ here describes the context on both sides.

To increase expressive power, the bi-directional RNN can be extended to a neural network of multiple RNN layers. For example, we can run deep RNNs in two directions and combine their results as in Eq. (4.18). Such model architectures have been extensively used in language and speech processing tasks, including machine translation [Wu et al., 2016], sentiment analysis [Tang et al., 2015], POS tagging [Huang et al., 2015], speech recognition [Graves et al., 2013a;b], and so on.

4.3 Memory

RNNs are appropriate for sequence learning as they effectively summarize past inputs at each step and base subsequent predictions on this historical summary. A key benefit of RNNs is that we can represent a history of arbitrary length as a fixed-size vector and update it dynamically when new information arrives. In other words, they use an implicit **memory** to store contextual

information. Next, we demonstrate that this memory mechanism is a general concept and can be used to improve sequence models.

4.3.1 Memory as A System

In psychology, memory is the cognitive ability of the mind to encode, store, and retrieve information. Among the many cognitive models in psychology, a foundational framework is the **multi-store model** [Atkinson and Shiffrin, 1968]. It defines memory as a system comprising three structural components: fleeting **sensory memory**, **short-term memory**, and **long-term memory**. Sensory memory retains raw stimuli that decay rapidly, such as immediate visual and olfactory data. Short-term memory stores information for a slightly longer duration but remains transient. For example, when attempting to memorize a phone number, the digits are held transiently and may be forgotten shortly after. Long-term memory, conversely, is permanent, retaining information indefinitely. For instance, adults can recall specific events from their childhood.

Given this categorization, there appear to be interesting connections between the above model of memory and the neural networks we discuss here. For example, the hidden state of a recurrent unit can be viewed as short-term memory: it maintains active information throughout a sequence but is entirely reset when switching to a new sequence⁷. On the other hand, the entire language model and its associated parameters function akin to a long-term memory: the model encodes global probabilistic word prediction patterns from the training corpus, retaining this knowledge permanently for future inference. Furthermore, standard cognitive metrics perfectly align with computational paradigms: **coding** or encoding is referred to as how the information is stored in a memory, **duration** is referred to as how long the information is stored in a memory, and **capacity** is referred to as how much information is stored in a memory.

In machine learning and NLP, we can gain an understanding of memory by considering it from an information processing point of view. Broadly, memory can be viewed as a system that writes information to a “storage” and reads it when queried. It has the following functions.

- **Encoding.** The input of the system is encoded in a form that is easy to process. For text inputs, this can be simply thought of as the same encoding process as we discuss in both Chapter 3 and this chapter: a word or a sequence of words is represented as a feature vector or a sequence of feature vectors.
- **Update.** Given the encoded information, we store it in the memory. This operation is generally dependent on the organization of the memory. For example, one can treat a group of encoded items as a datastore with an indexing system. In this case, storing an item requires finding the right place to keep it. Alternatively, one can represent the memory as a single vector of numbers.
- **Retrieval.** The stored information can be retrieved. This typically involves matching each item in the memory against an input query. If the memory is represented in a simpler form, such as a vector, it may not be explicitly retrieved, and we return the entire

⁷Alternatively, the hidden state at step i may simply retain very little information about the earliest steps due to information decay.

memory when required.

These functions can be designed in many different ways, leading to a variety of NLP systems. One simple example is information retrieval (IR) [Manning et al., 2008]. A typical IR system indexes a vast collection of documents and allows users to search for relevant information. To enable efficient search, documents are encoded into convenient representations. For example, a bag-of-words model might compute the matching score between a document and a query, while an inverted index efficiently maps query terms to their storage locations. Systems of this type cover a wide range of applications, including translation memory, dialogue, summarization, document classification, and so on.

Another design choice made for memory systems is to consider, either partially or fully, a continuous form for the above components. One method is to encode each input item as a real-valued vector (e.g., a word embedding) but use the same modules of update and retrieval as in usual information retrieval-like systems [Weston et al., 2015; Khandelwal et al., 2019]. An alternative method is to adopt differentiable functions for all the steps in building and accessing the memory. These models are typically implemented using neural networks and trained using gradient descent [Sukhbaatar et al., 2015; Graves et al., 2014; Kumar et al., 2016; Graves et al., 2016; Miller et al., 2016]. This idea motivates work on exploring approaches to coupling neural networks with memories, such as **end-to-end memory networks** and **neural Turing machines**. Note that the above models are sometimes called **external memories**, as they are used as separate modules working with other systems.

Memory can also work as an internal or hidden component of a system. In this case, the memory is typically rebuilt for each input sample, and so it can be regarded as an instance of the short-term memory. There are various ways of using this type of memory to improve sequence models. In the remainder of the section, we will focus on using the memory mechanism in RNNs. In Chapter 5, we will see how the idea of memory is extended to model the correspondence between tokens of two sequences.

4.3.2 Long Short-Term Memory

In the vanilla RNN presented in Section 4.2.1, the summarization of the context words was given by the output of a recurrent unit. It implicitly defines a memory, and thus in principle allows modeling dependencies over arbitrary lengths. The memory simply combines the representations of the earlier history $w_1 \cdots w_{i-1}$ and the input at the current step w_i , but does not consider how much information from different steps should be compressed into a fixed-length representation. A problem with this model is that, when predictions require long-term dependencies, the memory may retain insufficient relevant information, and it may be hard to learn these dependencies through back-propagation [Bengio et al., 1994; Pascanu et al., 2013]. A more powerful approach, therefore, is to compute what should be retained at each step, and to let the model learn to decide whether to memorize or forget.

Long short-term memory (LSTM) is perhaps the best-known variant of RNNs to accomplish the above goal [Hochreiter and Schmidhuber, 1997]. The basic idea of LSTM is that a recurrent unit can learn to memorize useful things and forget useless things by maintaining an

explicit memory [Gers et al., 2000]. To this end, the vanilla recurrent unit is replaced with an LSTM unit that is made up of an output vector (call it a **recurrent cell**), a memory vector (call it a **memory cell**), and three gates to control the information flow inside the LSTM unit. As an extension to RNNs, an LSTM network deals with an input sequence as usual: it starts with some initial states, and then repeatedly takes an input and outputs a vector. A key difference between LSTM networks and RNNs is that the LSTM unit of step i takes both the recurrent cell and the memory cell of its previous step. The form of an LSTM unit is given by:

$$(\mathbf{h}_i, \mathbf{c}_i) = \text{LSTM}(\mathbf{h}_{i-1}, \mathbf{c}_{i-1}, \mathbf{x}_i), \quad (4.19)$$

where $\mathbf{h}_i \in \mathbb{R}^{d_h}$ is the recurrent cell of step i , $\mathbf{c}_i \in \mathbb{R}^{d_h}$ is the memory cell of step i , and $\mathbf{x}_i \in \mathbb{R}^{d_e}$ is the input of step i . Given $\text{LSTM}(\cdot)$, applying the LSTM model is straightforward. We simply repeat the call of $\text{LSTM}(\cdot)$ for the inputs $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ and obtain the outputs $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$. This resembles the way we use vanilla RNNs, making it very easy to extend LSTM to multi-layer models (see Section 4.2.3) and bi-directional models (see Section 4.2.4).

We can divide $\text{LSTM}(\cdot)$ into three steps.

- **Step 1: Forget.** Assuming that $\mathbf{c}_{i-1}$ contains the information that the model memorizes at step $i-1$, we need to determine how much information in $\mathbf{c}_{i-1}$ is discarded in building $\mathbf{c}_i$. To do this, a gate is used to control to what extent we forget for each dimension of $\mathbf{c}_{i-1}$. The **forget gate** is defined to be:

$$\mathbf{f}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_f + \mathbf{x}_i\mathbf{V}_f + \mathbf{b}_f), \quad (4.20)$$

where $\mathbf{f}_i \in [0, 1]^{d_h}$ is a vector with the same number of dimensions as $\mathbf{c}_{i-1}$. The Sigmoid function maps the input data to the range $[0, 1]$. Thus, an entry of $\mathbf{f}_i$ indicates how much is preserved for the same entry of $\mathbf{c}_{i-1}$. Taking this further, $\mathbf{f}_i \odot \mathbf{c}_{i-1}$ represents the memory that is retained after passing through the forget gate. See Figure 4.5 (a) for an illustration of the forget gate in the LSTM unit.

- **Step 2: Update.** Next we update the memory by considering both the previous state of the memory (i.e., $\mathbf{c}_{i-1}$) and the input of the LSTM unit (i.e., $\mathbf{x}_i$ and $\mathbf{h}_{i-1}$). We first combine $\mathbf{x}_i$ and $\mathbf{h}_{i-1}$ using a simple neural network:

$$\hat{\mathbf{c}}_i = \text{TanH}(\mathbf{h}_{i-1}\mathbf{U}_c + \mathbf{x}_i\mathbf{V}_c + \mathbf{b}_c). \quad (4.21)$$

$\hat{\mathbf{c}}_i$ can be treated as the new information we intend to add to the memory at step i . Again, we need a way to control the amount of information coming into the memory. Hence we define an **input gate** as

$$\mathbf{g}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_g + \mathbf{x}_i\mathbf{V}_g + \mathbf{b}_g). \quad (4.22)$$

This equation is similar to Eq. (4.20) but with different parameters. We then define $\mathbf{g}_i \odot \hat{\mathbf{c}}_i$ to be the actual new information that we are interested in. Taking both $\mathbf{f}_i \odot \mathbf{c}_{i-1}$

and $\mathbf{g}_i \odot \hat{\mathbf{c}}_i$, the memory cell at step i is given by:

$$\mathbf{c}_i = \mathbf{f}_i \odot \mathbf{c}_{i-1} + \mathbf{g}_i \odot \hat{\mathbf{c}}_i. \quad (4.23)$$

In other words, we forget something old in $\mathbf{c}_{i-1}$ and memorize something new in $\hat{\mathbf{c}}_i$. See Figure 4.5 (b) for an illustration of the update step.

- **Step 3: Output.** In the last step we generate the output $\mathbf{h}_i$ based on the memory $\mathbf{c}_i$. Instead of copying $\mathbf{c}_i$ to $\mathbf{h}_i$, we feed $\mathbf{c}_i$ to the hyperbolic tangent function and multiply its result with the output gate. Like Eqs. (4.20) and (4.22), the output gate is given by:

$$\mathbf{o}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_o + \mathbf{x}_i\mathbf{V}_o + \mathbf{b}_o). \quad (4.24)$$

Then, the output of the LSTM unit is defined to be:

$$\mathbf{h}_i = \mathbf{o}_i \odot \text{TanH}(\mathbf{c}_i). \quad (4.25)$$

See Figure 4.5 (c) for an illustration of the output step.

The LSTM model is parameterized by $\mathbf{U}_f, \mathbf{U}_c, \mathbf{U}_g, \mathbf{U}_o \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V}_f, \mathbf{V}_c, \mathbf{V}_g, \mathbf{V}_o \in \mathbb{R}^{d_e \times d_h}$, and $\mathbf{b}_f, \mathbf{b}_c, \mathbf{b}_g, \mathbf{b}_o \in \mathbb{R}^{d_h}$. Compared with vanilla RNNs, additional parameters arise directly from this gate-based architecture. In practice one can implement LSTM units in many different ways, e.g., using activation functions other than $\text{Sigmoid}(\cdot)$ and $\text{TanH}(\cdot)$, removing the bias terms $\mathbf{b}_f$, $\mathbf{b}_c$, $\mathbf{b}_g$, and $\mathbf{b}_o$. Training LSTM models follows the standard paradigm of training RNN-based models. For example, we can build an LSTM-based language model and train it by using the methods presented in Section 4.2.2.

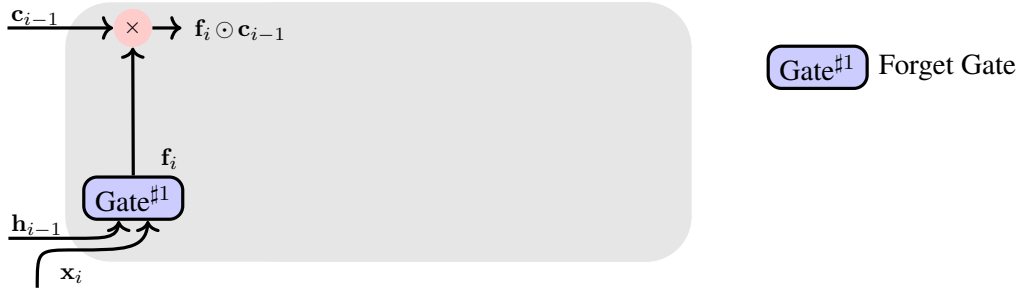
4.3.3 Gated Recurrent Units

Above, we saw the important role played by the gate units and the memory cell. In general the use of these neural networks makes the model computationally more expensive. A computationally efficient alternative to the LSTM is the **gated recurrent unit (GRU)**, which adopts a simplified model structure with fewer gate functions [Cho et al., 2014; Chung et al., 2014]. Unlike LSTM, a GRU does not have a memory cell so, as an RNN unit, it takes both the previous state vector $\mathbf{h}_{i-1}$ and the current input vector $\mathbf{x}_i$, and produces the current state vector $\mathbf{h}_i$.

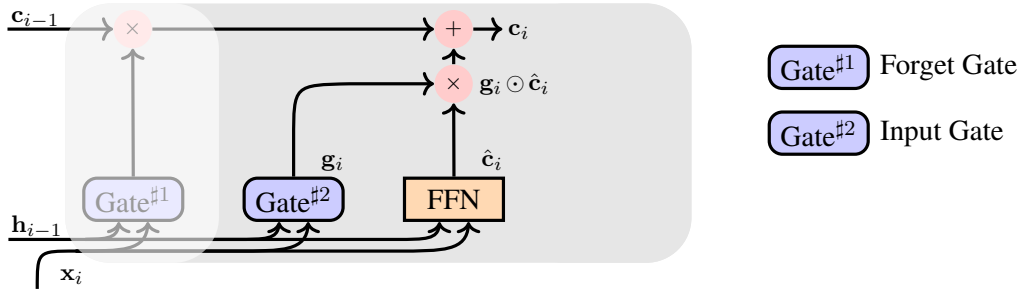
The GRU architecture incorporates two gate units: the **reset gate** and the **update gate**. The reset gate, as the name suggests, is used to reset (or rescale) the state of the GRU (i.e., $\mathbf{h}_{i-1}$). Analogous to the gates in an LSTM, the reset gate is defined to be:

$$\mathbf{r}_i = \text{Sigmoid}(\mathbf{h}_{i-1}\mathbf{U}_r + \mathbf{x}_i\mathbf{V}_r + \mathbf{b}_r), \quad (4.26)$$

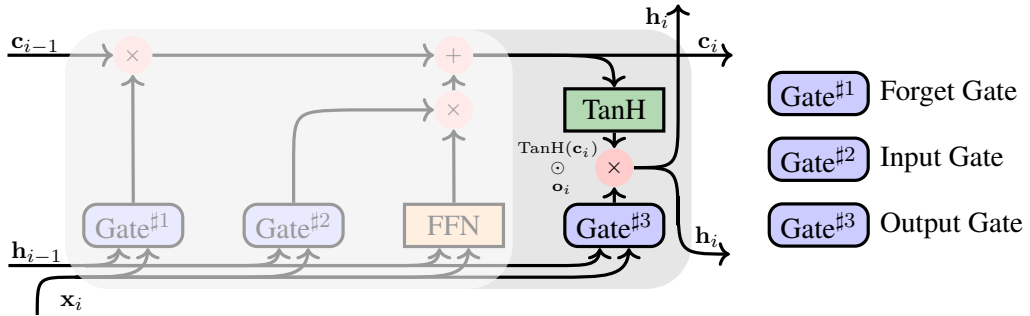
where $\mathbf{r}_i \in [0, 1]^{d_h}$ is a vector of scalars, each dimension describing how much information in the corresponding dimension of $\mathbf{h}_{i-1}$ is retained. Thus, we have a representation of retained



(a) The forget gate.



(b) The input gate.



(c) The output gate.

Figure 4.5: The architecture of the LSTM unit. At step i , it takes the input $\mathbf{x}_i$, and then updates both the memory cell ($\mathbf{c}_{i-1} \rightarrow \mathbf{c}_i$) and the recurrent cell ($\mathbf{h}_{i-1} \rightarrow \mathbf{h}_i$). This process involves three gates: the forget gate controls how much information in $\mathbf{c}_{i-1}$ is retained at step i , the input gate controls how much information in $\mathbf{c}_{i-1}$ and $\mathbf{x}_i$ is retained at step i , and the output gate controls how much information in $\mathbf{c}_i$ is used to form $\mathbf{h}_i$.

information

$$v_{i-1} = \mathbf{r}_i \odot \mathbf{h}_{i-1}. \quad (4.27)$$

Taking both the retained information v_{i-1} and the current input $\mathbf{x}_i$, a new state vector is defined

to be:

$$\hat{\mathbf{h}}_i = \text{TanH}(v_{i-1} \mathbf{U}_h + \mathbf{x}_i \mathbf{V}_h + \mathbf{b}_h). \quad (4.28)$$

The update gate is then given by:

$$\mathbf{u}_i = \text{Sigmoid}(\mathbf{h}_{i-1} \mathbf{U}_u + \mathbf{x}_i \mathbf{V}_u + \mathbf{b}_u), \quad (4.29)$$

$\mathbf{u}_i$ can be thought of as an interpolation weight vector that balances the preservation of the old state against the integration of the new candidate state. Finally, the output of the GRU is defined as a linear interpolation of $\hat{\mathbf{h}}_i$ and $\mathbf{h}_{i-1}$

$$\mathbf{h}_i = \mathbf{u}_i \odot \hat{\mathbf{h}}_i + (1 - \mathbf{u}_i) \odot \mathbf{h}_{i-1}. \quad (4.30)$$

Figure 4.6 illustrates the information flow in a GRU unit. The parameters here are $\mathbf{U}_r, \mathbf{U}_h, \mathbf{U}_u \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{V}_r, \mathbf{V}_h, \mathbf{V}_u \in \mathbb{R}^{d_e \times d_h}$, and $\mathbf{b}_r, \mathbf{b}_h, \mathbf{b}_u \in \mathbb{R}^{d_h}$. Therefore, the GRU model is smaller than the LSTM model because of the use of fewer gate units. Note that removing the memory cell makes GRUs more efficient. In this case, the role of memory is implicitly assumed by the hidden state $\mathbf{h}_i$, which dynamically updates to preserve important information over time.

4.4 Convolutional Models

In this section we describe another type of model for sequence modeling, called convolutional neural networks (CNNs). While our overview covers the basic concepts, it does not serve as an exhaustive introduction to various CNN variants and cutting-edge techniques. Instead, we focus on the application of CNNs to sequential data and present some architectural refinements.

4.4.1 Convolution

A key property of CNNs is their shared-weight architecture by which a kernel or filter slides over the input data and produces a feature map. The idea is that the filter only receives signals from a restricted region of data at a time (call it the **receptive field**), and computes the weighted sum of these input signals. To illustrate this, we consider the common case where a filter in CNNs is generally used to deal with 2D data. Consider a 3×3 data matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 9 & 7 \\ 3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}, \quad (4.31)$$

and a 2×2 filter with a weight matrix

$$\mathbf{W} = \begin{bmatrix} 2 & 0 \\ 2 & 2 \end{bmatrix}. \quad (4.32)$$

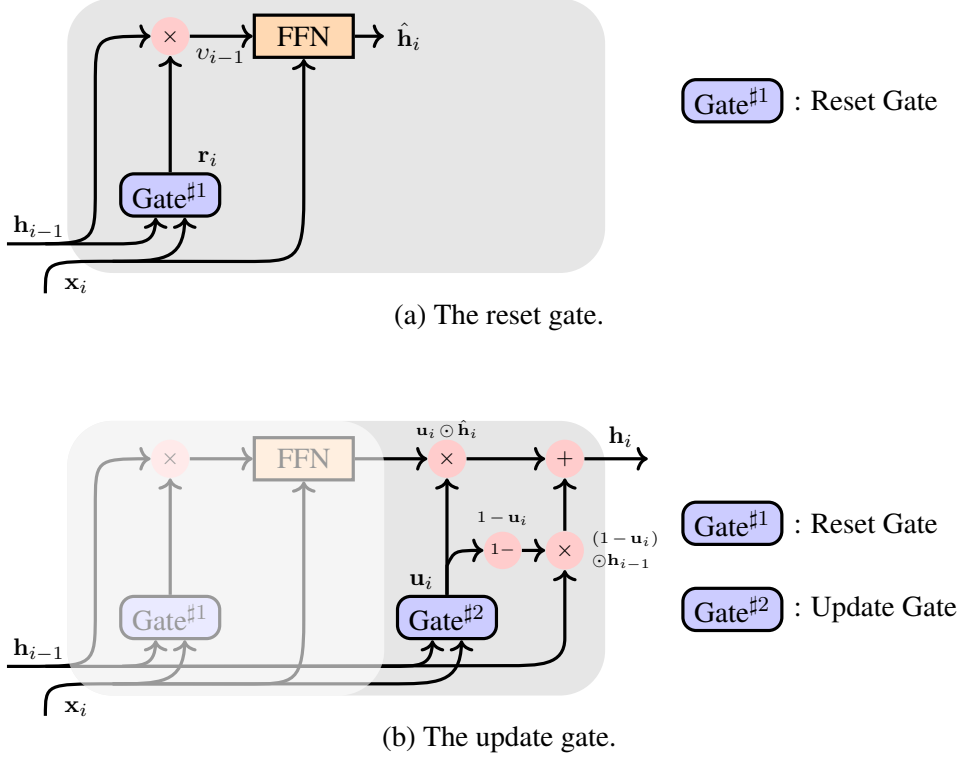


Figure 4.6: The architecture of the GRU. Unlike the LSTM unit, the GRU does not involve a memory cell, and thus follows the same input and output forms of a standard RNN unit. There are two gates in the GRU. The reset gate controls how much information in $\mathbf{h}_{i-1}$ is retained at step i . The retained information is then fused with the input $\mathbf{x}_i$, generating the candidate output $\hat{\mathbf{h}}_i$. The update gate seeks a balance between $\hat{\mathbf{h}}_i$ and $\mathbf{h}_{i-1}$ in computing the final output of the GRU.

We apply this filter to every 2×2 sub-matrix of $\mathbf{A}$ (yielding four such sub-matrices in this example) and compute the sum of the elements weighted by $\mathbf{W}$. For instance, considering the 2×2 sub-matrix in the upper-left corner of $\mathbf{A}$, the output of the filter is calculated as:

$$\begin{aligned}
 \text{Conv}\left(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \mathbf{W}\right) &= \text{Conv}\left(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 2 & 2 \end{bmatrix}\right) \\
 &= 1 \times 2 + 9 \times 0 + 3 \times 2 + 1 \times 2 \\
 &= 10.
 \end{aligned} \tag{4.33}$$

$\text{Conv}(\cdot)$ defines a **convolution operation** that sums the entries of the element-wise product of the two matrices. The convolution operation can be extended to cover the entire input matrix

by sliding the filter over it, as follows:

$$\begin{aligned}
 \text{Conv}(\mathbf{A}, \mathbf{W}) &= \text{Conv}\left(\begin{bmatrix} 1 & 9 & 7 \\ 3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}, \mathbf{W}\right) \\
 &= \begin{bmatrix} \text{Conv}\left(\begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}, \mathbf{W}\right) & \text{Conv}\left(\begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}, \mathbf{W}\right) \\ \text{Conv}\left(\begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}, \mathbf{W}\right) & \text{Conv}\left(\begin{bmatrix} 1 & 2 \\ 1 & -1 \end{bmatrix}, \mathbf{W}\right) \end{bmatrix} \\
 &= \begin{bmatrix} 10 & 24 \\ 8 & 2 \end{bmatrix}.
 \end{aligned} \tag{4.34}$$

The output $\begin{bmatrix} 10 & 24 \\ 8 & 2 \end{bmatrix}$ is also called the **feature map** for the filter $\mathbf{W}$ on $\mathbf{A}$. Sometimes, the convolution operation $\text{Conv}(\mathbf{A}, \mathbf{W})$ is written as $\mathbf{A} * \mathbf{W}$ where the symbol $*$ stands for the **convolution product**.⁸

Now let us consider a more general description of convolution in CNNs. Suppose that $\mathbf{A}$ is a multi-dimensional data array. A filter defines a window (or receptive field) on $\mathbf{A}$. We can move the window on $\mathbf{A}$ in different directions. This results in a set of data arrays, denoted by Ω . Each data array $\mathbf{a}_p \in \Omega$ is formed by the elements from the corresponding region of $\mathbf{A}$. For example, there are four sub-matrices in Eq. (4.34): $\mathbf{a}_1 = \begin{bmatrix} 1 & 9 \\ 3 & 1 \end{bmatrix}$, $\mathbf{a}_2 = \begin{bmatrix} 9 & 7 \\ 1 & 2 \end{bmatrix}$, $\mathbf{a}_3 = \begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}$, and $\mathbf{a}_4 = \begin{bmatrix} 1 & 2 \\ 1 & -1 \end{bmatrix}$. Also, we suppose the filter is parameterized by a weight array $\mathbf{W}$ of the same size as $\mathbf{a}_p$, i.e., $|\mathbf{a}_p| = |\mathbf{W}|$. The result of applying the filter to $\mathbf{A}$ is an array of features

$$\text{Conv}(\mathbf{A}, \mathbf{W}) = \begin{bmatrix} v_1 & \cdots & v_{|\Omega|} \end{bmatrix}. \tag{4.37}$$

⁸In mathematical analysis, given two integrable functions $f(\cdot)$ and $g(\cdot)$, **convolution** defines a new integrable function $f * g(\cdot)$ to describe the integral of $f(\cdot)$ weighted by reflected, shifted $g(\cdot)$. More formally, the convolution for continuous functions is defined as:

$$f * g(x) = \int_{\mathbb{R}} f(y)g(x-y)dy, \tag{4.35}$$

where $f(y)$ is the function that we are concerned with, and $g(x-y)$ is the weight function which is translated by reflecting $g(y)$ along the y -axis and then shifting it by x . A special case is that x and y are both integers. In this case, we can define $f * g(\cdot)$ as

$$f * g(x) = \sum_y f(y)g(x-y), \tag{4.36}$$

which is the basic form of Eq. (4.33). In CNNs, x , y and $x-y$ can be seen as indices of items in data arrays. $f(y)$ is a data item in the input array, and $g(x-y)$ is the corresponding weight in the filter. By using Eq. (4.36), we calculate the value of the item indexed by x in the output array $f * g(x)$ (i.e., the feature map).

Each feature v_p is given by:

$$\begin{aligned}
 v_p &= \text{Conv}(\mathbf{a}_p, \mathbf{W}) \\
 &= \mathbf{a}_p \cdot \mathbf{W} \\
 &= \sum_{k=1}^{|\mathbf{W}|} a_p(k) \cdot W(k),
 \end{aligned} \tag{4.38}$$

where $a_p(k)$ and $W(k)$ represent the k -th elements of the flattened arrays $\mathbf{a}_p$ and $\mathbf{W}$, respectively. Note that while the array $[v_1 \dots v_{|\Omega|}]$ can be reshaped into various formats (such as a matrix or a 3D tensor) based on the input dimensions, these representations are fundamentally equivalent from a data storage perspective. For instance, applying a 2D filter to 2D input data yields a 2D feature matrix, as demonstrated in Eq. (4.34).

Furthermore, we need to address two issues to make the model practical. First, we must specify the **stride**, which represents the step size of the filter as it slides over $\mathbf{A}$. In the previous example, we assumed a stride of 1 (i.e., $\text{stride} = 1$). By employing a larger stride, we effectively downsample $\mathbf{A}$, compressing it into a smaller number of features. Second, to control the spatial dimensions of the output feature map, we can introduce dummy elements (or **padding**) around the input data boundaries. Zero-padding, which surrounds the input region with zeros, is the most common approach. For example, consider the 2×2 data matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 9 \\ 7 & 3 \end{bmatrix}. \tag{4.39}$$

We can add zero-valued entries around it to obtain a 4×4 matrix

$$\mathbf{A}_{\text{padding}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 9 & 0 \\ 0 & 7 & 3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{4.40}$$

Using the same filter as in Eq. (4.33) with $\text{stride} = 1$, we have a 3×3 feature map

$$\text{Conv}_{\text{stride}=1}(\mathbf{A}_{\text{padding}}, \mathbf{W}) = \begin{bmatrix} 2 & 20 & 18 \\ 14 & 22 & 24 \\ 0 & 14 & 6 \end{bmatrix}. \tag{4.41}$$

If $\text{stride} = 2$, then we would have a feature map with the same size as the input data

$$\text{Conv}_{\text{stride}=2}(\mathbf{A}_{\text{padding}}, \mathbf{W}) = \begin{bmatrix} 2 & 18 \\ 0 & 6 \end{bmatrix}. \tag{4.42}$$

Note that strictly speaking, the operation implemented in deep learning is technically **cross-correlation** rather than true mathematical convolution. True convolution requires the

filter to be reflected (flipped) before sliding it over the input. However, because the filter weights in CNNs are learned from scratch, this flipping step is mathematically redundant. Most deep learning frameworks omit it for computational efficiency, though the field conventionally retains the term “convolution”.

4.4.2 CNNs for Sequence Modeling

Following the formulation in the previous sections, we assume that the input of a sequence model is a vector sequence $\mathbf{x}_1 \cdots \mathbf{x}_m$ and the output is another vector sequence $\mathbf{h}_1 \cdots \mathbf{h}_m$. For example, we can think of $\mathbf{x}_1 \cdots \mathbf{x}_m$ as a matrix $\mathbf{X} \in \mathbb{R}^{m \times d_e}$ in which the i -th row vector represents the i -th word of the sequence.

It is straightforward to perform convolution on $\mathbf{X}$. Since $\mathbf{x}_i$ does not impose an ordering across feature dimensions, it is unnecessary to apply convolution across feature dimensions. Hence, we can utilize a receptive field of size $r \times d_e$ and consider all dimensions of $\mathbf{x}_i$ simultaneously in the convolution operation. In practical applications, multiple filters are often employed to capture different aspects of the inputs. For instance, one might use a filter with a large receptive field to incorporate broader context, while using another with a small receptive field to concentrate on highly local features. Figure 4.7 illustrates the application of two distinct filters to an input sequence.

To distribute the extracted features to $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, we associate each application of a filter with a specific position in the sequence. To ensure the input and output sequences maintain the same length, a padding vector is added to both ends of the input sequence. The following table shows the input and output of a CNN for an example sequence:

Position	Input	Receptive Field	Output
0	$\mathbf{x}_0 (= 0)$	N/A	N/A
1	$\mathbf{x}_1$	$\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2\}$	$\mathbf{h}_1$
2	$\mathbf{x}_2$	$\{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$	$\mathbf{h}_2$
3	$\mathbf{x}_3$	$\{\mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4\}$	$\mathbf{h}_3$
4	$\mathbf{x}_4$	$\{\mathbf{x}_3, \mathbf{x}_4, \mathbf{x}_5\}$	$\mathbf{h}_4$
5	$\mathbf{x}_5 (= 0)$	N/A	N/A

An activation function is typically applied to introduce non-linearity to the final output. In this manner, we construct a standard convolutional layer, which can be viewed as a sequence of fully connected neural networks, each taking inputs from a fixed-size window. For the i -th position, the output of the convolutional layer is given by:

$$v_i = \psi(\text{Conv}(\mathbf{a}_i, \mathbf{W})), \quad (4.43)$$

where $\mathbf{a}_i$ represents the inputs in the receptive field⁹, and $\mathbf{W}$ represents the parameters of the filter. In situations involving multiple filters (e.g., d_h filters), we have a set of parame-

⁹For an $r \times d_e$ receptive field, $\mathbf{a}_i$ is defined to be $\{\mathbf{x}_{\lceil i - \frac{r}{2} \rceil}, \dots, \mathbf{x}_{\lceil i + \frac{r}{2} \rceil - 1}\}$ or $\{\mathbf{x}_{\lfloor i - \frac{r}{2} \rfloor}, \dots, \mathbf{x}_{\lfloor i + \frac{r}{2} \rfloor - 1}\}$.

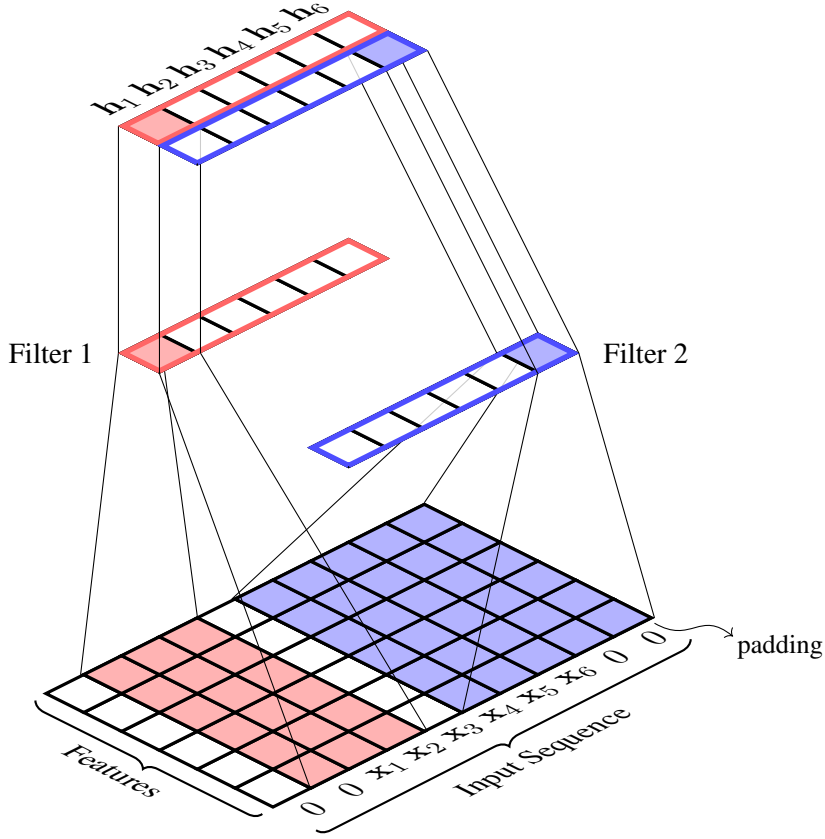


Figure 4.7: Two filters applied to a sequence of word vectors. The input involves ten word vectors (words $x_1 \cdots x_6$ and two padding words on each of the two ends of the sequence). Each word vector has 6 dimensions, and so, the input is a 10×6 matrix. Filter 1 has a receptive field of size 3×6 . By sliding it over the input matrix, we obtain a sequence of outputs, each corresponding to a position (i.e., a sequence of 8 outputs). Similarly, we apply filter 2 to the input sequence and obtain another sequence of outputs. The two output sequences are then organized as a 2×6 matrix in which the i -th row vector is $\mathbf{h}_i$.

ters $\{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(d_h)}\}$, a set of activation functions $\{\psi^{(1)}, \dots, \psi^{(d_h)}\}$, and a set of inputs $\{\mathbf{a}_i^{(1)}, \dots, \mathbf{a}_i^{(d_h)}\}$. Each tuple $(\mathbf{W}^{(k)}, \psi^{(k)}, \mathbf{a}_i^{(k)})$ gives an output by

$$v_i^{(k)} = \psi^{(k)}(\text{Conv}(\mathbf{a}_i^{(k)}, \mathbf{W}^{(k)})). \quad (4.44)$$

Note that $v_i^{(k)}$ is simply an entry of $\mathbf{h}_i$. Thus, $\mathbf{h}_i$ can be written as

$$\mathbf{h}_i = \begin{bmatrix} v_i^{(1)} & \dots & v_i^{(d_h)} \end{bmatrix}. \quad (4.45)$$

Many CNN-based systems of practical interest comprise two or more convolutional layers. The most straightforward approach to achieve this is through layer stacking, analogous to multi-layer RNNs (see Section 4.2.3). Specifically, the output sequence of a lower convolutional layer

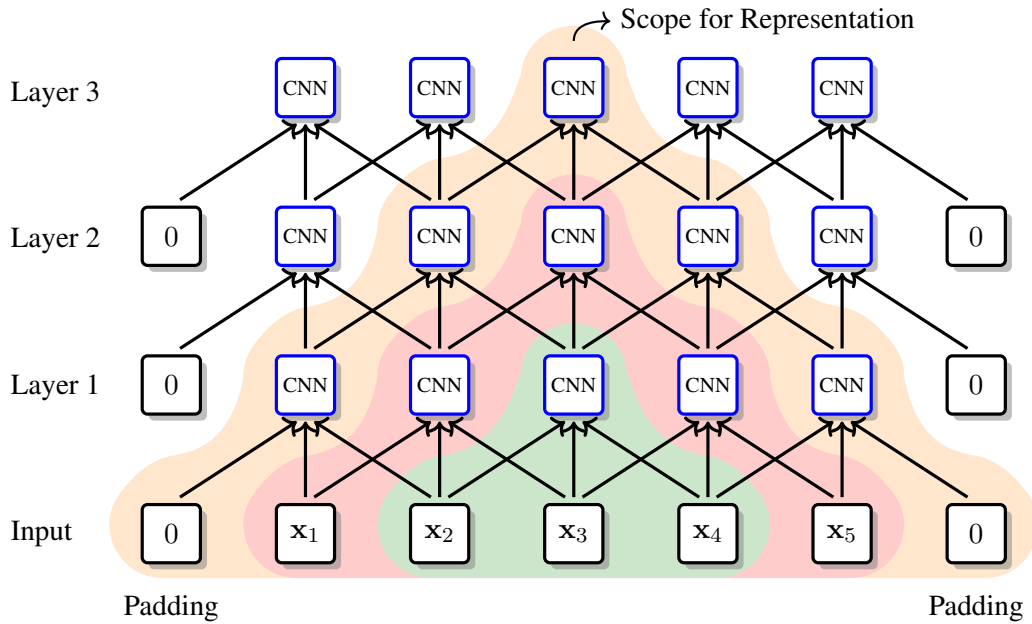


Figure 4.8: A CNN with 3 convolutional layers (stride = 1 and $r = 3$). Each layer takes a sequence of vectors and produces another sequence of vectors. In this process a filter moves over the input and performs the convolution operation in each move. In layer 1, the receptive field of the filter is a region of three input items (see green shadows). The higher a layer is, the larger receptive field a filter has. For example, in layer 3, an application of the filter can at most cover the entire input sequence (see orange shadows).

serves directly as the input to the subsequent layer. Figure 4.8 provides an example of a CNN architecture involving three stacked convolutional layers. A primary advantage of multi-layer CNNs is the exponential expansion of their receptive fields, leading to a broader scope for representation. As illustrated in Figure 4.8, while a neuron in Layer 1 aggregates information strictly from three input vectors, a neuron in Layer 3 indirectly aggregates information spanning seven input vectors. Because neurons in higher layers process signals from a much larger contiguous span of the original sequence, they are capable of extracting higher-level semantic representations and effectively capturing long-distance dependencies.

In some applications, we need a fixed-length, low-dimensional representation of the entire sequence. A common way is to add a pooling layer to merge $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ into a single vector. For example, we can select the maximum value (i.e., max-pooling) or average the values (i.e., average-pooling) along each dimension. This is a generic method in machine learning and is applicable to most of the sequence models discussed in this book.

4.4.3 Handling Positional Information

One compelling property of CNNs is their ability to balance complexity and efficiency by restricting full connectivity to a localized region of the input data. This local connectivity

allows us to model a convolutional layer mathematically similarly to a fully-connected layer: the output of a neuron is a non-linear transformation of the weighted sum of its inputs. Despite the computational efficiency of this local connectivity, a limitation of standard CNNs is their translation invariance. While they capture local structures within the receptive field, they lack an awareness of absolute global positions within the sequence.

Interestingly, if we restrict ourselves to sequence modeling, this might seem unproblematic because the output of the model is itself a sequence. It seems reasonable to assume that the output sequence preserves the ordering information of the input sequence. However, applying CNNs to sequential data does not guarantee a strict one-to-one mapping between the input and output items. Technically, $\mathbf{h}_i$ is not simply a representation of $\mathbf{x}_i$. Rather, it encodes a window of inputs centered around $\mathbf{x}_i$. This significantly complicates the problem, as it becomes exceedingly difficult to infer from $\mathbf{h}_i$ exactly how those localized inputs were originally ordered.

Explicitly modeling word orders is very important in NLP, and has been extensively studied in tasks like machine translation [Lopez, 2008; Koehn, 2010]. For neural network-based models, one may address the problem by resorting to order-sensitive model architectures like RNNs. A more popular approach in recent systems is to develop a **positional encoding** sub-model and incorporate it into existing sequence models [Gehring et al., 2017b; Vaswani et al., 2017; Shaw et al., 2018; Dufter et al., 2022]. Formally, we say that the input at position i is a combination of the original input $\mathbf{x}_i$ and the positional encoding of i (denoted by $\text{PE}(\cdot)$):

$$\mathbf{e}_i = \text{Merge}(\mathbf{x}_i, \text{PE}(i)), \quad (4.46)$$

where $\text{Merge}(\cdot)$ defines a combination operation. The use of positional encoding is straightforward: one simply needs to replace $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ with $\{\mathbf{e}_1, \dots, \mathbf{e}_m\}$ in a sequence model. So, this approach is model-agnostic.

In this subsection, we present several versions of $\text{PE}(\cdot)$ and ways to combine them with $\mathbf{x}_i$. Note that the following discussion is not specific to CNNs. We consider it here because positional encoding is useful for models that are insensitive to the order of inputs, and CNNs are a good example to see how it is used [Gehring et al., 2017a;b]. In Chapter 6, we will see an application of positional encoding in Transformer which is a state-of-the-art neural model in many areas.

1. Offset-based Positional Encoding

The simplest way to represent a position i is to use its absolute index. This can be formulated as the “distance” from a reference point

$$\text{PE}(i) = i - i_0, \quad (4.47)$$

where i_0 is an integer indicating where we start counting. If $i_0 = 0$, $\text{PE}(i) = i$ yields the standard absolute position. Note that $\text{PE}(i)$ could be a negative number if $i_0 > i$. While not a physical distance in the strict sense, it remains a perfectly valid feature for machine learning algorithms. To enforce a non-negative measure, the right-hand side of Eq. (4.47) can

be bounded using an absolute value:

$$\text{PE}(i) = |i - i_0|. \quad (4.48)$$

Treating positions as simple integers leads to unbounded, discrete positional encoding. A more desirable method might be to use a continuous representation in a range of values, because it allows the system to work within a sample space that is smooth and easy to optimize. A simple way to do this is normalization. For example, dividing $i - i_0$ by some maximum value, we obtain a normalized version of the offset-based encoding

$$\text{PE}(i) = \frac{i - i_0}{i_{\max} - i_0}. \quad (4.49)$$

For example, we can set $i_{\max}$ = the maximum possible length of the sequence and $i_0 = 0$, so that $\text{PE}(i)$ chooses its value in $[0, 1]$. Another common choice is to set $i_{\max} = m$ (i.e., the length of the input sequence) and define $\text{PE}(i)$ as a ratio whose value varies as m changes.

To make use of these scalar positions, it is straightforward to enrich the original input vectors by adding new dimensions, provided they can be viewed as new features. Thus, $\mathbf{e}_i$ is given by:

$$\mathbf{e}_i = [\mathbf{x}_i, \text{PE}(i)], \quad (4.50)$$

where $[\cdot]$ stands for the concatenation operation.

2. Sinusoidal Positional Encoding

The next obvious step is to represent positions as vectors instead of scalars. Although vectorizing the representations of positions sounds complicated, a simple idea is to use a carrying system which describes how a natural number is expressed by a polynomial with respect to a base [Kerns, 2021]. For example, i can be written as

$$i = \sum_{k=0}^{k_{\max}} a(i, k) b^k, \quad (4.51)$$

where $a(i, k)$ is the k -th digit, $k_{\max} + 1$ is the maximum number of digits, and b is the base of the system. The carrying occurs when $a(i, k)$ reaches b : we increase $a(i, k + 1)$ by 1 and roll back $a(i, k)$ to 0. In this way we can change $a(i, k)$ with a period of b^k , that is, $a(i, 0)$ changes with a period of b^0 , $a(i, 1)$ changes with a period of b^1 , $a(i, 2)$ changes with a period of b^2 , and so on.

Using this system, i can be represented as a vector

$$\text{PE}(i) = \begin{bmatrix} a(i, 0) & a(i, 1) & \cdots & a(i, k_{\max}) \end{bmatrix}. \quad (4.52)$$

For example, when $b = 2$, $\text{PE}(11) = \begin{bmatrix} 1 & 1 & 0 & 1 \end{bmatrix}$. However, in Eq. (4.52), $\text{PE}(i)$ is still a discrete function. As discussed throughout this book, we may want a continuous vector

representation that can describe intermediate states between discrete events. Considering $a(i, k)$ as a periodic function, a common choice is the **sine** function. Thus $a(i, k)$ can be re-defined, as follows:

$$a(i, k) = \sin(i \cdot \omega_k). \quad (4.53)$$

This function has an amplitude of 1 and a period of $\frac{2\pi}{\omega_k}$. Using an analogous form of periods to that used in Eq. (4.51), we define ω_k as

$$\omega_k = \frac{1}{(b_{\text{model}})^{k/d_{\text{model}}}}, \quad (4.54)$$

where $b_{\text{model}} > 0$ and $d_{\text{model}} > 0$ are hyper-parameters of the model. Clearly, we have $\frac{2\pi}{\omega_0} < \frac{2\pi}{\omega_1} < \dots < \frac{2\pi}{\omega_{k_{\text{max}}}}$.

Similarly, we can define $a(i, k)$ via the **cosine** function

$$a(i, k) = \cos(i \cdot \omega_k). \quad (4.55)$$

Taking both Eqs. (4.53) and (4.55), we create a new representation of i , as follows:

$$\text{PE}(i) = \begin{bmatrix} \sin(i \cdot \omega_0) & \cos(i \cdot \omega_0) & \dots & \sin(i \cdot \omega_{k_{\text{max}}}) & \cos(i \cdot \omega_{k_{\text{max}}}) \end{bmatrix}. \quad (4.56)$$

Vaswani et al. [2017] instantiated the above form by setting $b_{\text{model}} = 10,000$. Let $\text{PE}(i, k)$ be the k -th dimension of $\text{PE}(i)$. Vaswani et al. [2017]’s version of positional encoding is written as

$$\text{PE}(i, 2k) = \sin\left(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}\right), \quad (4.57)$$

$$\text{PE}(i, 2k+1) = \cos\left(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}\right). \quad (4.58)$$

Choosing $b_{\text{model}} = 10,000$ is empirical. One can adjust it for specific tasks. Figure 4.9 plots the positional encoding for different positions. We see that, when k becomes larger, the change of the color follows a larger period.

Note that Eqs. (4.57) and (4.58) have a useful property that $\text{PE}(i + \mu)$ can be easily

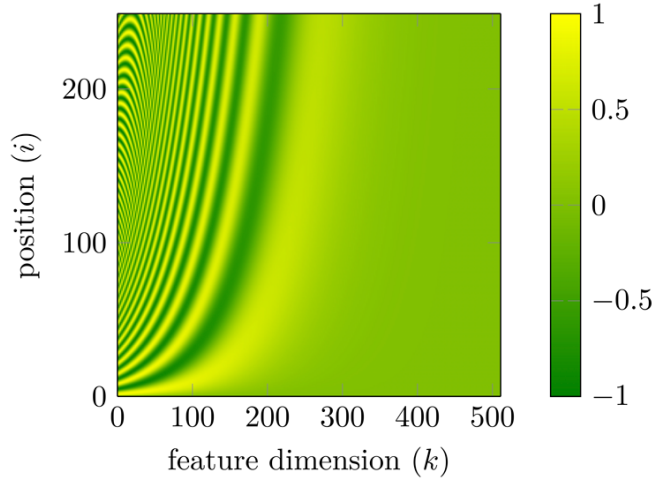


Figure 4.9: A heat map of the positional embedding model of Eqs. (4.57) and (4.58) ($b_{\text{model}} = 10,000$ and $d_{\text{model}} = 512$). Consider a position i (i.e., the i -th row), then move another position j from i upwards or downwards. Intuitively, when i and j are closer, the corresponding row vectors are more similar. By contrast, when j moves away from i , the similarity is not that obvious. This property helps explain the idea behind the positional embedding model: the “distance” between two positions is implicitly modeled by comparing their multi-dimensional representations.

expressed by a linear function of $\text{PE}(i)$ for a given offset μ ¹⁰

$$\begin{aligned} \text{PE}(i + \mu, 2k) &= \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k + 1) + \\ &\quad \text{PE}(i, 2k + 1) \cdot \text{PE}(\mu, 2k), \end{aligned} \quad (4.61)$$

$$\begin{aligned} \text{PE}(i + \mu, 2k + 1) &= \text{PE}(i, 2k + 1) \cdot \text{PE}(\mu, 2k + 1) - \\ &\quad \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k). \end{aligned} \quad (4.62)$$

This property is advantageous because it allows the model to attend to relative positions. That is, the state at position $i + \mu$ can be described by starting with i and then appending it with the offset μ .

When applying the sinusoidal positional encoding, one way is to use Eq. (4.50) to concatenate $\mathbf{x}_i$ and $\text{PE}(i)$. In Vaswani et al. [2017]’s work, they instead assume $\text{PE}(i)$ to be a vector of the same size as $\mathbf{x}_i$ (i.e., $|\text{PE}(i)| = |\mathbf{x}_i| = d_e$), and add $\text{PE}(i)$ to $\mathbf{x}_i$

$$\mathbf{e}_i = \mathbf{x}_i + \text{PE}(i). \quad (4.63)$$

¹⁰One can derive these by taking

$$\sin(\alpha + \beta) = \sin(\alpha) \cdot \cos(\beta) + \cos(\alpha) \cdot \sin(\beta), \quad (4.59)$$

$$\cos(\alpha + \beta) = \cos(\alpha) \cdot \cos(\beta) - \sin(\alpha) \cdot \sin(\beta). \quad (4.60)$$

This sinusoidal additive model has been the basis of many positional encoding approaches [Dehghani et al., 2018; Likhomanenko et al., 2021; Su et al., 2021].

3. Learnable Positional Encoding

The application of sinusoidal positional encoding yields a lookup table $\mathbf{C}_{\text{PE}} \in \mathbb{R}^{m_{\text{max}} \times d_e}$ (where m_{max} is the maximum sequence length)

$$\mathbf{C}_{\text{PE}} = \begin{bmatrix} \text{PE}(1) \\ \vdots \\ \text{PE}(m_{\text{max}}) \end{bmatrix}. \quad (4.64)$$

In this table, each row vector $\text{PE}(i)$ corresponds to the embedding of a position i . These vectors, as described previously, these sinusoidal vectors are deterministic and derived from fixed mathematical functions. An alternative approach is to treat vectors of positions as parameters of the model and learn them as usual. In this case, both word embeddings and position embeddings can be trained in the same manner. See Chapters 2 and 3 for more information about learning word embeddings in neural language models.

One last note on positional encoding. What we have shown in this section can broadly be characterized as an **absolute positional encoding** paradigm: a position is described by its location in a coordinate system. Another concept that is worth exploring is **relative positional encoding** [Shaw et al., 2018]. For example, we can extend Eq. (4.48) to define the distance between two positions i and j

$$\text{PE}(i, j) = |i - j|. \quad (4.65)$$

In this case, the positional encoding is no longer an attribute of the i -th input but some representation of the distance relative to a reference position j . In fact, most of the methods for relative positional encoding are variants on a theme in which positions are described by their pair-wise relationships. This relational inductive bias forms the foundation of several advanced architectures, which will be extensively detailed in Chapters 6 and 8.

4.5 Examples

Both recurrent and convolutional models have been successfully used in numerous applications. Here we discuss a few of the interesting examples. While these models are mostly basic, they form the foundations of many state-of-the-art systems.

4.5.1 Text Classification

To illustrate how sequence models can be used, we first consider the text classification task, where we assign a predefined class to a given text sequence. This framework naturally extends to a broad spectrum of NLP problems, including categorizing news articles, analyzing sentiment, identifying spam emails, detecting deceptive comments, and so forth.

In text classification we are interested in predicting the class label from a set C , given a word sequence $w_1 \cdots w_m$:

$$\hat{c} = \arg \max_{c \in C} \text{Score}(c, w_1 \cdots w_m). \quad (4.66)$$

Here $\text{Score}(c, w_1 \cdots w_m)$ measures how well a class c is predicted for the input sequence $w_1 \cdots w_m$. We map the sequence of words to the sequence of word vectors (or word embeddings), that is, $w_1 \cdots w_m \rightarrow \mathbf{x}_1 \cdots \mathbf{x}_m$. Assuming $\text{Score}(\cdot)$ is a probabilistic function that describes the distribution of the classes, we can reformulate the problem as

$$\begin{aligned} \hat{c} &= \arg \max_{c \in C} \Pr(c | w_1 \cdots w_m) \\ &= \arg \max_{c \in C} \Pr(c | \mathbf{x}_1 \cdots \mathbf{x}_m). \end{aligned} \quad (4.67)$$

The central issue here is the modeling of $\Pr(c | \mathbf{x}_1 \cdots \mathbf{x}_m)$. We define $\Pr(c | \mathbf{x}_1 \cdots \mathbf{x}_m)$ by following the general encoder-predictor framework, as follows

- The input $\mathbf{x}_1 \cdots \mathbf{x}_m$ is represented as a feature vector $\mathbf{H} \in \mathbb{R}^{d_h}$ by using an encoder

$$\mathbf{H} = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \quad (4.68)$$

- $\mathbf{H}$ is fed to a standard Softmax layer to predict the class distribution

$$\Pr(\cdot | \mathbf{H}) = \text{Softmax}(\mathbf{H} \cdot \mathbf{U} + \mathbf{b}), \quad (4.69)$$

where $\mathbf{U} \in \mathbb{R}^{d_h \times |C|}$ and $\mathbf{b} \in \mathbb{R}^{|C|}$ are model parameters.

The $\text{Encode}(\cdot)$ component is the same as the architectures discussed in the preceding sections. As a result, a wide variety of sequence encoding models are directly applicable here. For example, consider the CNN-based encoder presented in Kim [2014]’s work. Kim [2014]’s model is based on a single convolution layer involving d_h filters. The application of a filter produces a set of features, each being associated with a position of the sequence. Since we want a single vector for representing the entire sequence, a pooling layer is added so that the number of features for each filter is reduced to one. Then, for any c , the probability $\Pr(c | \mathbf{H})$ can be computed directly according to Eq. (4.69). See Figure 4.10 for an illustration of this classifier.

Training such a model involves optimizing a predefined loss function, and, as mentioned several times in this book, one of the most common methods is to minimize the cross-entropy loss using gradient descent. Also, we can use regularization to improve the training of CNNs. More details about these techniques can be found in Chapter 2.

It is worth noting that while this baseline model is straightforward, it remains one of the most highly effective architectures for text classification. Many architectural refinements exist. Prominent extensions of this framework include deep CNNs [Conneau et al., 2017b], character-level CNNs [Santos and Gatti, 2014; Zhang et al., 2015], and recurrent-convolutional

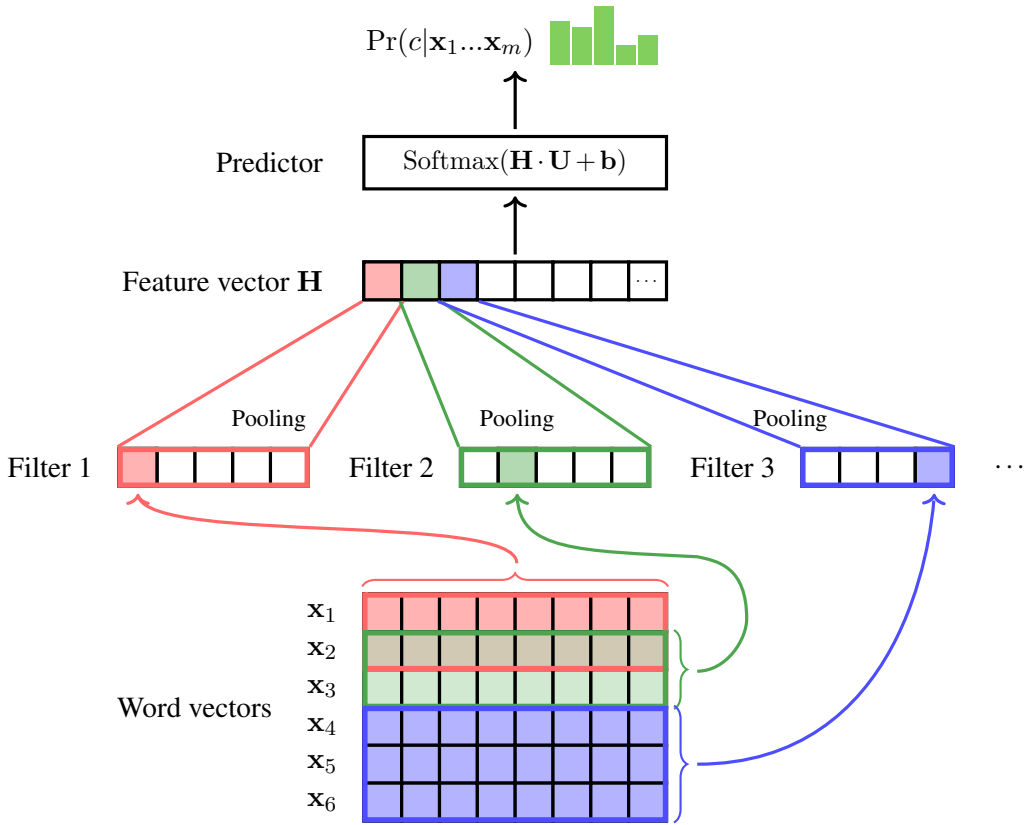


Figure 4.10: A CNN-based text classifier [Kim, 2014]. The input is a sequence of word vectors. A convolutional layer involving multiple filters is used to extract features in different dimensions. A pooling layer is used to reduce the number of features for representing the input text, leading to a low-dimensional feature vector $\mathbf{H}$. The prediction conditions on $\mathbf{H}$ and is made by using a standard Softmax layer.

hybrids [Lai et al., 2015].

4.5.2 End-to-End Speech Recognition

Speech recognition is the task of mapping a sequence of acoustic signals to a sequence of words or characters (called a **transcription**) [Reddy, 1976; Rabiner and Juang, 1993]. Since the original input is an acoustic waveform over the time domain, it is standard practice to transform it into a sequence of waveform fragments (called **frames**). Typically, a frame is represented as a feature vector, denoted by $\mathbf{x}_i$. This is achieved by employing either engineered feature functions from signal processing [Davis and Mermelstein, 1980; Picone, 1993; Campbell, 1997] or learnable neural embeddings [Chorowski et al., 2019; Schneider et al., 2019]. Regarding the output, speech recognition systems generally do not directly produce full words. Instead, they output sequences of **transcription units** (or **transcription labels**), such as phonemes, characters, or sub-words. In this subsection, we assume that the output of the system is a sequence of characters, denoted by $y_1, \dots, y_n \in V_y^n$. The vocabulary

V_y consists of standard English letters (a – z), digits (0 – 9), spaces ($\langle \text{sp} \rangle$), periods ($\langle \text{pe} \rangle$), and other punctuation marks. As with most modern speech recognition systems, we add a blank symbol ϵ to the alphabet in order to indicate the null output.

The goal here is to find a string $\hat{y}_1 \cdots \hat{y}_n$ for a given input sequence $\mathbf{x}_1 \cdots \mathbf{x}_m$, so that

$$\hat{y}_1 \cdots \hat{y}_n = \arg \max_{y_1 \cdots y_n} \Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m). \quad (4.70)$$

This problem is more challenging than the classification problem described in the previous subsection, as the output can be an arbitrary string, rather than a class in a predefined class set. The string generation problem leads to two difficulties. First, we need some mechanism to model $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m)$ for an exponentially large number of pairs of input and output sequences. Second, in practice $y_1 \cdots y_n$ is often much shorter than $\mathbf{x}_1 \cdots \mathbf{x}_m$ (i.e., $n < m$), and so we need some mechanism to align a long sequence to a short one. However, we do not need to consider these difficulties in the stage of representing the input sequence, and can still encode the input sequence $\mathbf{x}_1 \cdots \mathbf{x}_m$ in the same way as other sequence models. Specifically, we represent $\mathbf{x}_1 \cdots \mathbf{x}_m$ in the following form

$$\mathbf{h}_1 \cdots \mathbf{h}_m = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \quad (4.71)$$

The encoder can be RNNs, CNNs, or more advanced models (such as Transformers). Here we consider the encoder architecture used in [Graves et al. \[2013b\]](#) and [Graves and Jaitly \[2014\]](#)’s work. It is a multi-layer bi-directional LSTM model. We skip the details of this model without loss of continuity, as the reader may already be familiar with it in Sections 4.2.3, 4.2.4 and 4.3.2.

Having obtained the sequence representation $\mathbf{H} = \mathbf{h}_1 \cdots \mathbf{h}_m$, a Softmax layer is used to map each $\mathbf{h}_i \in \mathbb{R}^{d_h}$ to a distribution of transcription labels, given by:

$$\Pr(\cdot | \mathbf{h}_i) = \text{Softmax}(\mathbf{h}_i \cdot \mathbf{U} + \mathbf{b}), \quad (4.72)$$

where $\mathbf{U} \in \mathbb{R}^{d_h \times |V_y|}$ and $\mathbf{b} \in \mathbb{R}^{|V_y|}$ are model parameters. $\Pr(\cdot | \mathbf{h}_i) \in \mathbb{R}^{|V_y|}$ is a probability distribution over V_y , and the probability of transcription label l_i at position i is simply $\Pr(l_i | \mathbf{h}_i)$. We can then write the probability of a label sequence in the form

$$\Pr(l_1 \cdots l_m | \mathbf{H}) = \prod_{i=1}^m \Pr(l_i | \mathbf{h}_i). \quad (4.73)$$

We can apply the $\arg \max$ operation to find the most probable label sequence as usual. However, $l_1 \cdots l_m$ cannot be straightforwardly used as a system output, because it often contains many duplicate and blank symbols. To post-process $l_1 \cdots l_m$, we first collapse consecutive identical labels into a single label, and then remove the blank symbols. For example, consider a label sequence

s s ϵ e e ϵ ϵ e $\langle \text{sp} \rangle$ y ϵ o o u.

By merging “s s”, “e e”, and “o o”, we have

$$s \ \epsilon \ e \ \epsilon \ \epsilon \ e \ \langle \text{sp} \rangle \ y \ \epsilon \ o \ u.$$

Then, we remove all ϵ and obtain

$$s \ e \ e \ \langle \text{sp} \rangle \ y \ o \ u.$$

The above sequence is what we would call a transcription. Clearly, different label sequences can correspond to the same transcription. Let $B(y_1 \cdots y_n)$ be the set of label sequences corresponding to $y_1 \cdots y_n$ ¹¹. We now turn to the following form of the transcription probability (see Figure 4.11 for an illustration)

$$\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m) = \sum_{l_1 \cdots l_m \in B(y_1 \cdots y_n)} \Pr(l_1 \cdots l_m | \mathbf{H}). \quad (4.74)$$

A problem with this model is that the number of sequences in $B(y_1 \cdots y_n)$ grows exponentially with n (and m). Fortunately, there are very efficient methods for computing $\sum_{l_1 \cdots l_m \in B(y_1 \cdots y_n)} \Pr(l_1 \cdots l_m | \mathbf{H})$. See [Graves et al., 2006] for a dynamic programming algorithm for solving this problem.

Note that Eq. (4.74) is also known as a form of **connectionist temporal classification (CTC)** [Graves et al., 2006]. It is one of the most widely used methods for training end-to-end speech recognition and speech translation systems. One of the merits of CTC is that it allows us to align any label sequence to a transcription in a very simple and efficient way. It is easy to make use of CTC in training a speech recognition system. Suppose there is a set of pairs of input sequence and transcription, denoted by $\mathcal{D}$. A common training objective is to maximize the likelihood of these transcriptions given the corresponding inputs, written as

$$\hat{\theta} = \arg \max_{\theta} \sum_{(y_1 \cdots y_n, \mathbf{x}_1 \cdots \mathbf{x}_m) \in \mathcal{D}} \log \Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m, \theta), \quad (4.75)$$

where $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m, \theta)$ denotes the probability computed via Eq. (4.74), and θ denotes the model parameters.

When testing on new data, we search for an optimal transcription as in Eq. (4.70). This process, also known as **decoding**, generally involves optimized search algorithms and pruning techniques. For example, we can use the 1-best label sequence instead of all possible label sequences to obtain an approximation to Eq. (4.74), that is, $\Pr(y_1 \cdots y_n | \mathbf{x}_1 \cdots \mathbf{x}_m) = \max \Pr(l_1 \cdots l_m | \mathbf{H})$. This leads to an efficient decoding method, called **Viterbi decoding**, which has been extensively used in speech recognition and machine translation [Lopez, 2008]. For more details about the decoding of sequence generation, we refer the reader to Chapter 5.

¹¹ $B(y_1 \cdots y_n)$ may contain label sequences of arbitrary lengths. However, if we restrict input to the sequence $\mathbf{x}_1 \cdots \mathbf{x}_m$, then each sequence in $B(y_1 \cdots y_n)$ is of length m .

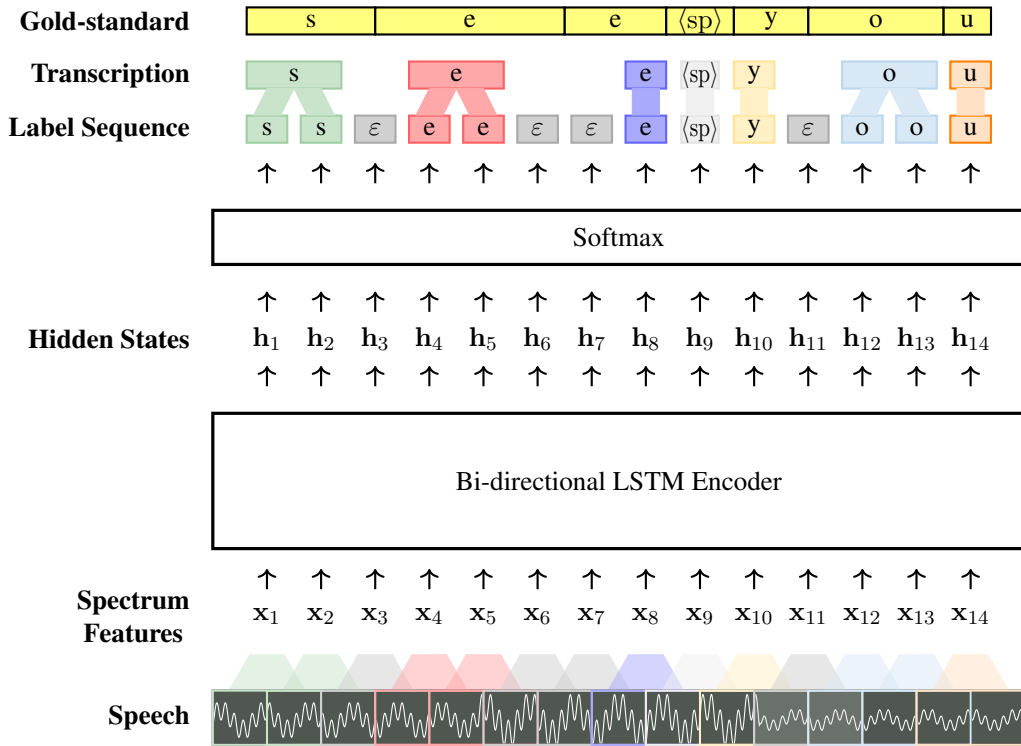


Figure 4.11: An end-to-end speech recognition architecture. The input of the system is a sequence of acoustic signals that are represented as a sequence of feature vectors (i.e., $x_1 \cdots x_{14}$). These feature vectors are taken by a bi-directional LSTM encoder. The output of the encoder is a sequence of contextualized representations (i.e., $h_1 \cdots h_{14}$) which is then fed into a Softmax layer for generating a distribution of labels at each position. We can then generate a sequence of labels from these distributions. We map each label sequence to a form of final output by eliminating duplicate symbols and blank symbols. An output of the system corresponds to multiple label sequences, and the probability of the output is the sum of the probabilities of these label sequences.

4.5.3 Sequence Labeling with LSTM and Graphical Models

Sequence labeling is a conceptually straightforward approach to labeling sequence data. In NLP, it has been widely used in many sub-areas, such as word segmentation, part-of-speech tagging, and chunking. Learning in these models involves predicting a label from a predefined label set V_y at each position of a sequence. Ideally, we wish to predict the optimal label sequence given the entire input, formulated as:

$$\hat{y}_1 \cdots \hat{y}_m = \arg \max_{y_1 \cdots y_m} \Pr(y_1 \cdots y_m | x_1 \cdots x_m), \quad (4.76)$$

where $x_1 \cdots x_m$ is an input sequence (such as a sequence of word vectors), and $y_1 \cdots y_m$ is a label sequence in which each label y_i corresponds to an input item x_i .

As we have seen in this chapter, Eq. (4.76) fits well within the formulation of the sequence modeling problem. As a first step, we use an encoder to map $x_1 \cdots x_m$ to a sequence of

contextualized representations, as follows:

$$\mathbf{h}_1 \cdots \mathbf{h}_m = \text{Encode}(\mathbf{x}_1 \cdots \mathbf{x}_m). \quad (4.77)$$

We define $\text{Encode}(\cdot)$ as a bi-directional LSTM model because it involves a memory mechanism for modeling long-range dependencies in both left and right contexts. Hence, the architecture of the encoder is the same as that used in the preceding subsection.

$\mathbf{h}_1 \cdots \mathbf{h}_m$ can then serve as the input to a standard sequence labeling system (see Figure 4.12). The sequence labeling problem has been discussed in Chapter 1, and many models can be applied to address it. The simplest is a classifier (such as maximum entropy models and SVM-based models) that predicts a label distribution for each $\mathbf{h}_i$. A limitation of these models is that the predictions are made independently of one another. A more powerful approach is to use **graphical models** to consider dependencies among predicted labels. For example, **hidden Markov models (HMMs)** describe how a sequence of observations (i.e., $\mathbf{x}_1 \cdots \mathbf{x}_m$) is generated given a sequence of variables (i.e., $y_1 \cdots y_m$). The key idea is to rewrite $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ by making independence assumptions to factorize the distribution, and approximate $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ by a product of simple factors. However, these models require probability density functions of continuous variables (e.g., $\Pr(\mathbf{x}_i | y_i)$) which are difficult to estimate. This significantly differentiates the use of HMMs in neural models from that in conventional models where all states and observations are discrete variables¹².

HMMs and their descendants can be viewed as instances of generative models. Another class of models commonly used for sequence labeling is discriminative models. One such model is **conditional random fields (CRFs)** [Lafferty et al., 2001]. The CRF model is characterized by its ability to directly model the conditional probability of a label sequence given the input sequence, which facilitates the integration of arbitrary features. Consider, for example, the **linear-chain CRF** [Sutton and McCallum, 2012]. It defines $\Pr(y_1 \cdots y_m | \mathbf{h}_1 \cdots \mathbf{h}_m)$ in the

¹²In HMMs, a sequence of variables can be viewed as a path of transiting over some states whose values are chosen from a pre-defined set. In each transition from one state to another, something is observed (call it an observation). By making some assumptions, we can approximate $\Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m)$ in the following fashion

$$\begin{aligned} \Pr(y_1 \cdots y_m | \mathbf{x}_1 \cdots \mathbf{x}_m) &= \frac{\Pr(y_1 \cdots y_m) \cdot \Pr(\mathbf{x}_1 \cdots \mathbf{x}_m | y_1 \cdots y_m)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)} \\ &\approx \frac{\prod_{i=1}^m \Pr(y_i | y_{i-1}) \cdot \prod_{i=1}^m \Pr(\mathbf{x}_i | y_i)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)} \\ &= \frac{\prod_{i=1}^m \Pr(y_i | y_{i-1}) \Pr(\mathbf{x}_i | y_i)}{\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)}, \end{aligned} \quad (4.78)$$

where $\Pr(y_i | y_{i-1})$ is the **transition probability** of moving from y_{i-1} to y_i (when $i = 1$, we define $\Pr(y_i | y_{i-1}) = \Pr(y_1 | y_0) = \Pr(y_1)$), and $\Pr(\mathbf{x}_i | y_i)$ is the **emission probability** of observing $\mathbf{x}_i$ given y_i . As the denominator $\Pr(\mathbf{x}_1 \cdots \mathbf{x}_m)$ is a constant for different $y_1 \cdots y_m$, it can be dropped in the argmax operation of Eq. (4.76), as follows:

$$\hat{y}_1 \cdots \hat{y}_m = \arg \max_{y_1 \cdots y_m} \prod_{i=1}^m \Pr(y_i | y_{i-1}) \Pr(\mathbf{x}_i | y_i). \quad (4.79)$$

To estimate $\Pr(\mathbf{x}_i | y_i)$, a possible solution is to take $\Pr(\mathbf{x}_i | y_i) = \frac{\Pr(y_i | \mathbf{x}_i) \Pr(\mathbf{x}_i)}{\Pr(y_i)}$, and use a neural network to compute $\Pr(y_i | \mathbf{x}_i)$.

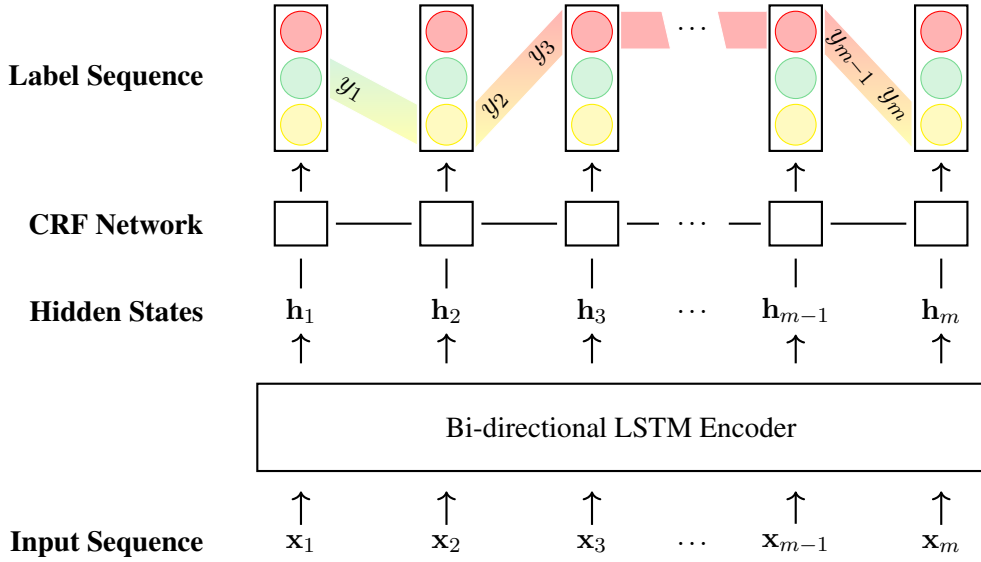


Figure 4.12: The BiLSTM + graphical model architecture for sequence labeling. The encoder is a standard bi-directional LSTM model. Given a sequence of input feature vectors (i.e., $x_1 \dots x_m$), it produces a new sequence of feature vectors for mapping the input to contextualized representations (i.e., $h_1 \dots h_m$). A CRF network is placed on $h_1 \dots h_m$ to predict a distribution of label sequences. The optimal label sequence is the one that has the maximum probability.

following form

$$\begin{aligned} \Pr(y_1 \dots y_m | h_1 \dots h_m) &= \frac{\Pr(y_1 \dots y_m, h_1 \dots h_m)}{\Pr(h_1 \dots h_m)} \\ &= \frac{\exp(\text{Score}(y_1 \dots y_m, h_1 \dots h_m))}{Z(h_1 \dots h_m)}, \end{aligned} \quad (4.80)$$

where $Z(h_1 \dots h_m)$ is a normalization factor, and has the form

$$Z(h_1 \dots h_m) = \sum_{y'_1 \dots y'_m} \exp(\text{Score}(y'_1 \dots y'_m, h_1 \dots h_m)). \quad (4.81)$$

$\text{Score}(\cdot)$ is a score for weighting the sequence pair $(y_1 \dots y_m, h_1 \dots h_m)$. It is given by summing over the values of a set of feature functions $\{f_1(\cdot), \dots, f_J(\cdot)\}$

$$\text{Score}(y_1 \dots y_m, h_1 \dots h_m) = \sum_{i=1}^m \sum_{j=1}^J f_j(y_i, y_{i-1}, h_i). \quad (4.82)$$

The outer loop of the summation corresponds to a visit to each position i . Given i , each function $f_j(\cdot)$ takes the current label y_i , the previous label y_{i-1} and the current input vector h_i , and then returns the value of a feature.

This model is called *linear-chain* because it assumes a first-order Markov property: each

label y_i is dependent only on its immediate neighbors y_{i-1} and y_{i+1} , conditioned on the observed sequence $\mathbf{H}$ ¹³, as follows:

$$\begin{array}{ccccccc}
 y_1 & \text{---} & \cdots & \text{---} & y_{i-1} & \text{---} & y_i & \text{---} & y_{i+1} & \text{---} & \cdots & \text{---} & y_m \\
 | & & & & | & & | & & | & & & & | \\
 \mathbf{h}_1 & & \cdots & & \mathbf{h}_{i-1} & & \mathbf{h}_i & & \mathbf{h}_{i+1} & & \cdots & & \mathbf{h}_m
 \end{array}$$

In CRFs, it is assumed that the variables in the graph depend only on their neighboring variables. Therefore, $f_j(y_i, y_{i-1}, \mathbf{h}_i)$ can be defined according to how y_i is connected. There are generally two types of features.

- **Transition-like Features.** These features model the connection between consecutive labels (y_{i-1}, y_i) , given by:

$$f_1(y_i, y_{i-1}, \mathbf{h}_i) = u(y_{i-1}, y_i), \quad (4.83)$$

where $u(y_{i-1}, y_i)$ is an entry of a weight matrix $\mathbf{u}$, indexed by (y_{i-1}, y_i) .

- **Emission-like Features.** The second type of feature models the connection between a label y_i and the associated input $\mathbf{x}_i$, given by:

$$f_2(y_i, y_{i-1}, \mathbf{h}_i) = g_i(y_i), \quad (4.84)$$

where $g_i(y_i)$ is the entry y_i of a vector $\mathbf{g}_i \in \mathbb{R}^{|V_y|}$. The vector $\mathbf{g}_i$ represents the weights of associating $\mathbf{h}_i$ with each label in the form

$$\mathbf{g}_i = \mathbf{h}_i \cdot \mathbf{v}, \quad (4.85)$$

where $\mathbf{v} \in \mathbb{R}^{d_h \times |V_y|}$ is a weight matrix.

To simplify notation, we use y_i (or y_{i-1}) to denote the one-hot representation for a label¹⁴. Then, substituting the above feature functions into Eq. (4.82) allows the scoring function to be written in the form

$$\begin{aligned}
 \text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m) &= \sum_{i=1}^m u(y_{i-1}, y_i) + g_i(y_i) \\
 &= \sum_{i=1}^m y_{i-1} \cdot \mathbf{u} \cdot y_i^\top + \mathbf{h}_i \cdot \mathbf{v} \cdot y_i^\top \\
 &= \sum_{i=1}^m (y_{i-1} \cdot \mathbf{u} + \mathbf{h}_i \cdot \mathbf{v}) \cdot y_i^\top.
 \end{aligned} \quad (4.86)$$

The right-hand side of the equation only involves simple algebraic operations on vectors

¹³CRFs can broadly be categorized as a type of **undirected graphical models**. They define a graph over a set of observed variables and a set of unobserved variables. These variables are connected in some way that forms a graph.

¹⁴In this case, $y_i \in \mathbb{R}^{|V_y|}$, although it is originally used as a scalar.

and matrices, allowing this model to be viewed as a standard neural network. In this way, it is convenient to implement the sequence labeling system with various neural network toolkits. We just need to stack a CRF network on an encoder network and learn the entire network as usual. For example, one can train this system by maximum likelihood, and optimize the loss function by gradient descent. Note that, as with other chain or lattice-based models, the CRF network can be efficient because there are dynamic programming algorithms, called the forward-backward methods, for computing both $\text{Score}(y_1 \cdots y_m, \mathbf{h}_1 \cdots \mathbf{h}_m)$ and $Z(\mathbf{h}_1 \cdots \mathbf{h}_m)$. We refer the interested reader to related papers for more detailed discussions [Lafferty et al., 2001; Sutton and McCallum, 2012].

One primary advantage of integrating distributed representations with sequence labeling is that it eliminates the need for manual feature engineering. Instead, the model autonomously learns features from raw input that are most effective for optimizing the sequence labeling objective. Such an architecture has been used as the backbone model for several state-of-the-art systems [Huang et al., 2015; Lample et al., 2016; Ma and Hovy, 2016; Li et al., 2020].

4.5.4 Hybrid Models for Language Modeling

As we have already noted, many sequence modeling problems can be handled by either RNN-based or CNN-based models. Each of these two architectures has its own characteristics: RNNs were originally designed to process variable-length temporal data, whereas CNNs are more effective at capturing local information in restricted regions of the input. Here, we consider a hybrid approach to language modeling to leverage the benefits of both.

Recall from Section 4.2.1 that a neural language model is trained to predict a probability distribution over a vocabulary, given some representation of the preceding words. Let $w_1 \cdots w_m$ be a word sequence whose probability we want to compute. First, we represent each word w_i as a word vector $\mathbf{x}_i$. Then, an RNN model processes one word vector at a time and outputs the probability:

$$\begin{aligned} \Pr(w_{i+1} | w_1 \cdots w_i) &= \Pr(w_{i+1} | \mathbf{x}_1 \cdots \mathbf{x}_i) \\ &= \Pr(w_{i+1} | \mathbf{h}_i), \end{aligned} \tag{4.87}$$

where $\mathbf{h}_i$ is the state of the recurrent unit at step i .

The process of converting words from symbols to continuous representations plays an important role in this model. While it is common for practitioners to obtain $\mathbf{x}_i$ from a word embedding table, this approach treats each word as a whole and simply ignores its internal structure. As a result, it might be difficult to learn distinct vectors for rare words in languages with large vocabularies [Bojanowski et al., 2017].

Here we consider a different way of representing words. The idea is simple: an additional neural network is used to embed words [Ling et al., 2015; Kim et al., 2016]. Suppose every word w_i can be expressed as a sequence of characters. We represent these characters as real-valued vectors $\mathbf{z}_{i,1} \cdots \mathbf{z}_{i,\text{len}_i}$ via a character embedding table. Following Kim et al. [2016]’s

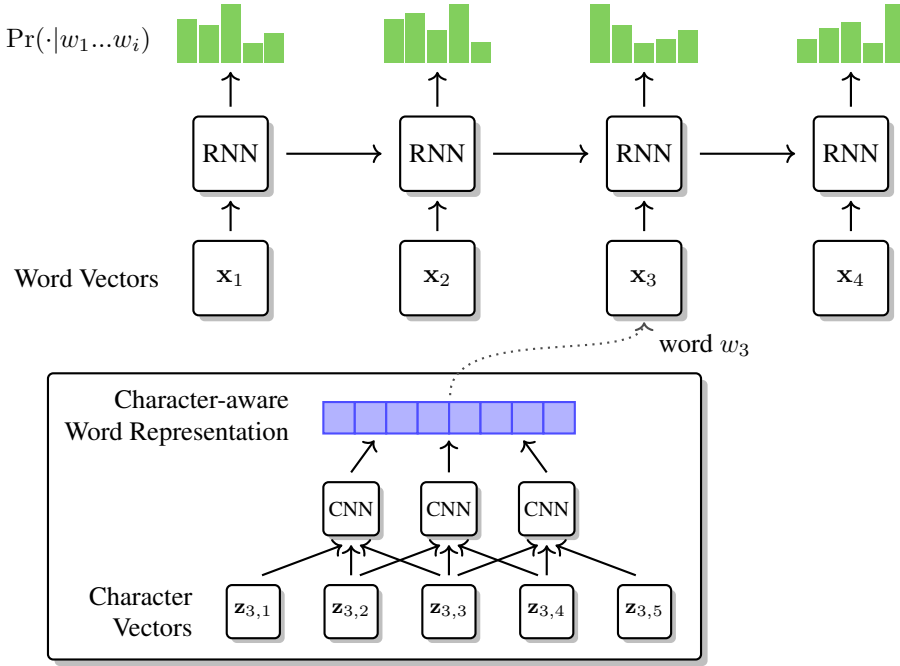


Figure 4.13: A language model with character-aware word representations [Kim et al., 2016]. As a language model, the goal of this model is to compute the probability $\text{Pr}(w_{i+1} | w_1 \dots w_i)$ for each i . We represent each word w_i as a real-valued vector $\mathbf{x}_i$. This vector is the output of a CNN that takes a sequence of characters corresponding to this word. Then, the sequence of the word vectors $\mathbf{x}_1 \dots \mathbf{x}_m$ is used as the input to an RNN + Softmax model. The model outputs at each position i a distribution of words, where the entry corresponding to w_{i+1} gives $\text{Pr}(w_{i+1} | w_1 \dots w_i)$. This hierarchical structure provides a multi-scale approach to language modeling: a sentence is modeled by considering words, and a word is modeled by considering characters.

work, we can use a CNN to represent $\mathbf{z}_{i,1} \dots \mathbf{z}_{i,\text{len}_i}$ as a word vector in the following form

$$\begin{aligned} \mathbf{x}_i &= \text{CNN}(\mathbf{z}_{i,1} \dots \mathbf{z}_{i,\text{len}_i}, \mathbf{W}) \\ &= \text{Pooling}(\text{TanH}(\text{Conv}(\mathbf{z}_{i,1} \dots \mathbf{z}_{i,\text{len}_i}, \mathbf{W}))), \end{aligned} \quad (4.88)$$

where $\text{Conv}(\cdot, \mathbf{W})$ is a convolutional layer with parameters $\mathbf{W}$, $\text{TanH}(\cdot)$ is a hyperbolic tangent function, and $\text{Pooling}(\cdot)$ is a pooling layer.

Figure 4.13 shows the architecture of the model. This model exhibits a hierarchical structure, in which characters form words, and words form sentences or phrases. From a practical perspective, in many NLP tasks it is quite natural to consider the hierarchical nature of language. We will see a few examples of making use of the relationships between different levels of language representations in later chapters.

4.6 Summary

This chapter has introduced the recurrent and convolutional neural network approaches to modeling sequences of words. On one hand, recurrent neural networks are designed for handling sequential data, and have broad applicability in NLP. To improve the modeling power of these models, memory mechanisms are commonly used. In particular, we have introduced LSTM and GRU which are two popular models for long sequences. On the other hand, while convolutional neural networks are commonly used to process visual data, they can be directly applied to sequence modeling. We have seen that all these models can be used in several language and speech processing tasks, including text classification, speech recognition, sequence labeling, and language modeling.

The roots of modeling sequences of language units can be traced back to early work in several different fields. For example, the process of generating a sequence of words can be described as a Markov chain where the prediction of a word only depends on a limited number of previous words [Markov, 1913]. This idea motivates the n -gram methods for sequence modeling [Shannon, 1948], as well as hidden Markov models which later appeared and became popular in modeling sequences of pairs of observed and unobserved variables [Baum and Petrie, 1966; Baum et al., 1970]. These models and their variants laid the foundations of many successful NLP systems in past decades [Manning and Schütze, 1999; Jurafsky and Martin, 2008].

The idea of using neural networks in sequence modeling has also been investigated for some time. One example illustrating how neural networks have been developed and applied to sequence modeling is speech recognition [Lippmann, 1989]. Most of the studies in the early days of this research area tried to either combine neural networks with existing models [Bourlard and Wellekens, 1990; Bourlard and Morgan, 1993; Trentin and Gori, 2001], or address sub-problems of speech recognition [Tank and Hopfield, 1987; Waibel et al., 1989; Lang et al., 1990; Bengio, 1991]. However, scaling up neural networks was challenging because training deep neural networks requires a lot of computational resources and data. The field had long been dominated by approaches based on hidden Markov models and **Gaussian mixture models (GMMs)**, with a pipeline of several modules that require careful tuning. On the other hand, while fully neural approaches were not state-of-the-art during that time, researchers were aware of their potential in learning representations of acoustic inputs and freeing them from hand-crafted features [LeCun and Bengio, 1995].

A dramatic shift from conventional pipelined approaches to end-to-end approaches came with the revival of neural networks in the 2000s [Hannun et al., 2014; Graves et al., 2013b]. This shift was so influential that it brought a wide range of fields together like never before. For instance, in both computer vision and speech processing, the past decade has witnessed tremendous performance gains driven by very deep neural networks and end-to-end learning [Hinton et al., 2006; Graves et al., 2013b; He et al., 2016; Krizhevsky et al., 2017]. In NLP, the paradigm shift started with the work on word embeddings [Mikolov et al., 2013; Pennington et al., 2014], and continues as more powerful sequence representation models are developed [Vaswani et al., 2017]. Simple approaches to sequence modeling, though not

discussed in depth in this chapter, are **compositional models** [Janssen, 2012]. For example, we can use the bag-of-words model to sum or average word vectors of a sequence. Despite the simple architectures of these approaches, they achieve satisfactory results in many tasks, providing strong baselines for further research on more advanced models [Conneau et al., 2018]. As the next step, applying recurrent and convolutional neural models to sequence modeling is straightforward. This is not surprising because these models are fairly well studied in other fields [Lipton et al., 2015; Li et al., 2021b; Khan et al., 2020]. In particular, the LSTM model is well suited to deal with long sequences and thus of great interest to NLP researchers [Sundermeyer et al., 2012; Huang et al., 2015; Wu et al., 2016]. However, research in this area is ongoing. Sequence modeling is one of the most active research fields with no end in sight. There are many models that are based on new architectures and show stronger performance in various tasks. More discussions on some of these models can be found in Chapters 6, 7 and 8.

Note that the term *sequence modeling* is currently used in many different ways, referring to different tasks. In many cases it is more common to use the terms *encoding* and *encoder* to emphasize the process of mapping a sequence of symbols to a continuous representation. As discussed in the previous sections, a benefit of viewing encoding as an individual task is that we can learn a general representation model that is not dependent on where we apply it. It opens the door to a wide range of pre-trained encoders for learning to represent various types of data, such as text [Peters et al., 2018; Devlin et al., 2019], speech [Oord et al., 2018; Hsu et al., 2021; Chen et al., 2022], vision [Chen and He, 2021; Bao et al., 2021; He et al., 2022], and combinations of these modalities [Chuang et al., 2020; Li et al., 2021a; Kim et al., 2021]. A closely related concept to text encoding is **text embedding** or **sentence embedding** [Conneau et al., 2017a; Cer et al., 2018]. These can be broadly considered equivalent. In general, an embedding model in NLP means a process of transforming the input text into a single low-dimensional vector rather than producing sequences of contextualized vectors [Kiros et al., 2015; Hill et al., 2016].

In many NLP problems, systems are not necessarily sequential on their input and/or output. For example, in text classification, a system may take tree-structured input and produce a label [Tai et al., 2015; Yang et al., 2016]. In this case we need some mechanism to encode hierarchical structures. An alternative approach is to convert trees to sequences (or **linearized trees**) so that we can directly make use of sequence models to handle non-sequential data [Vinyals et al., 2015]. This is an appealing idea because it opens up the possibility of developing a universally applicable encoder to represent various types of data if the input of the encoder can be linearized in some way. For example, by representing an image as a sequence of patches, sequence models can be directly applied to image classification, achieving state-of-the-art results on several tasks [Chen et al., 2020; Dosovitskiy et al., 2021].

Bibliography

- [Atkinson and Shiffrin, 1968] Richard C Atkinson and Richard M Shiffrin. Human memory: A proposed system and its control processes. In *Psychology of learning and motivation*, volume 2, pages 89–195. Elsevier, 1968.
- [Bao et al., 2021] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. Beit: Bert pre-training of image transformers. In *Proceedings of International Conference on Learning Representations*, 2021.
- [Baum and Petrie, 1966] Leonard E Baum and Ted Petrie. Statistical inference for probabilistic functions of finite state markov chains. *The annals of mathematical statistics*, 37(6):1554–1563, 1966.
- [Baum et al., 1970] Leonard E Baum, Ted Petrie, George Soules, and Norman Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of markov chains. *The annals of mathematical statistics*, 41(1):164–171, 1970.
- [Bengio, 1991] Yoshua Bengio. *Artificial neural networks and their application to sequence recognition*. PhD thesis, McGill University, 1991.
- [Bengio et al., 1994] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.
- [Bishop, 2006] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [Bojanowski et al., 2017] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the association for computational linguistics*, 5:135–146, 2017.
- [Bourlard and Wellekens, 1990] Herve Bourlard and Christian J Wellekens. Links between markov models and multilayer perceptrons. *IEEE Transactions on pattern analysis and machine intelligence*, 12(12):1167–1178, 1990.
- [Bourlard and Morgan, 1993] Herve A. Bourlard and Nelson Morgan. *Connectionist Speech Recognition: A Hybrid Approach*. Kluwer Academic Publishers, USA, 1993.
- [Box et al., 2015] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control (4th ed.)*. John Wiley & Sons, 2015.
- [Campbell, 1997] Joseph P Campbell. Speaker recognition: A tutorial. *Proceedings of the IEEE*, 85(9):1437–1462, 1997.
- [Cer et al., 2018] Daniel Cer, Yinfei Yang, Sheng yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Yun-Hsuan Sung, Brian Strope, and Ray Kurzweil. Universal sentence encoder. *arXiv preprint arXiv:1803.11175*, 2018.
- [Chatfield, 2003] Chris Chatfield. *The analysis of time series: an introduction*. Chapman and hall/CRC,

- 2003.
- [Chen et al., 2020] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International conference on machine learning*, pages 1691–1703. PMLR, 2020.
- [Chen et al., 2022] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, and Furu Wei. Wavlm: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.
- [Chen and He, 2021] Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15750–15758, 2021.
- [Cho et al., 2014] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [Chorowski et al., 2019] Jan Chorowski, Ron J Weiss, Samy Bengio, and Aäron Van Den Oord. Unsupervised speech representation learning using wavenet autoencoders. *IEEE/ACM transactions on audio, speech, and language processing*, 27(12):2041–2053, 2019.
- [Chuang et al., 2020] Yung-Sung Chuang, Chi-Liang Liu, Hung-yi Lee, and Lin-shan Lee. Speechbert: An audio-and-text jointly learned language model for end-to-end spoken question answering. In *Proceedings of Interspeech 2020*, pages 4168–4172, 2020.
- [Chung et al., 2014] Junyoung Chung, Çağlar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *Proceedings of NIPS 2014 Workshop on Deep Learning, December 2014*, 2014.
- [Conneau et al., 2017] Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. Supervised learning of universal sentence representations from natural language inference data. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, 2017a.
- [Conneau et al., 2017] Alexis Conneau, Holger Schwenk, Loïc Barrault, and Yann Lecun. Very deep convolutional networks for text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*, pages 1107–1116, 2017b.
- [Conneau et al., 2018] Alexis Conneau, Germán Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. What you can cram into a single vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126–2136, 2018.
- [Davis and Mermelstein, 1980] Steven Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE transactions on acoustics, speech, and signal processing*, 28(4):357–366, 1980.
- [Dehghani et al., 2018] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.

- [Devlin et al., 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.
- [Dosovitskiy et al., 2021] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of ICLR 2021*, 2021.
- [Dufter et al., 2022] Philipp Dufter, Martin Schmitt, and Hinrich Schütze. Position information in transformers: An overview. *Computational Linguistics*, 48(3):733–763, 2022.
- [Elman, 1990] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.
- [Fuller, 2009] Wayne A Fuller. *Introduction to statistical time series*. John Wiley & Sons, 2009.
- [Gehring et al., 2017] Jonas Gehring, Michael Auli, David Grangier, and Yann Dauphin. A convolutional encoder model for neural machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 123–135, 2017a.
- [Gehring et al., 2017] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N Dauphin. Convolutional sequence to sequence learning. In *International conference on machine learning*, pages 1243–1252. PMLR, 2017b.
- [Gers et al., 2000] Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural computation*, 12(10):2451–2471, 2000.
- [Goodfellow et al., 2016] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [Graves and Jaitly, 2014] Alex Graves and Navdeep Jaitly. Towards end-to-end speech recognition with recurrent neural networks. In *Proceedings of International conference on machine learning*, pages 1764–1772, 2014.
- [Graves et al., 2006] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, pages 369–376, 2006.
- [Graves et al., 2013] Alex Graves, Navdeep Jaitly, and Abdel-rahman Mohamed. Hybrid speech recognition with deep bidirectional lstm. In *IEEE workshop on automatic speech recognition and understanding*, pages 273–278. IEEE, 2013a.
- [Graves et al., 2013] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE international conference on acoustics, speech and signal processing*, pages 6645–6649. IEEE, 2013b.
- [Graves et al., 2014] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [Graves et al., 2016] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwinska, Sergio Gomez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538

- (7626):471–476, 2016.
- [Hamilton, 1994] James Douglas Hamilton. *Time Series Analysis*. Princeton University Press, 1994.
- [Hannun et al., 2014] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, and Adam Coates. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.
- [He et al., 2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [He et al., 2022] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.
- [Hill et al., 2016] Felix Hill, Kyunghyun Cho, and Anna Korhonen. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1367–1377, 2016.
- [Hinton et al., 2006] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.
- [Hochreiter and Schmidhuber, 1997] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [Hsu et al., 2021] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.
- [Huang et al., 2015] Zhiheng Huang, Wei Xu, and Kai Yu. Bidirectional lstm-crf models for sequence tagging. *arXiv preprint arXiv:1508.01991*, 2015.
- [Ioffe and Szegedy, 2015] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.
- [Jaeger and Haas, 2004] Herbert Jaeger and Harald Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *science*, 304(5667):78–80, 2004.
- [Janssen, 2012] Theo M.V. Janssen. Compositionality: its historic context. In M. Werning, W. Hinzen, and E. Machery, editors, *The Oxford handbook of compositionality*. Oxford University Press, 2012.
- [Jurafsky and Martin, 2008] Dan Jurafsky and James H. Martin. *Speech and Language Processing (2nd ed.)*. Prentice Hall, 2008.
- [Kernes, 2021] Jonathan Kernes. Master positional encoding: Part i, 05 2021. URL <https://towardsdatascience.com/master-positional-encoding-part-i-63c05d90a0c3>.
- [Khan et al., 2020] Asifullah Khan, Anabia Sohail, Umme Zahoor, and Aqsa Saeed Qureshi. A survey of the recent architectures of deep convolutional neural networks. *Artificial intelligence review*, 53(8):5455–5516, 2020.
- [Khandelwal et al., 2019] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike

- Lewis. Generalization through memorization: Nearest neighbor language models. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2019.
- [Kim et al., 2021] Wonjae Kim, Bokyung Son, and Ildoo Kim. Vilt: Vision-and-language transformer without convolution or region supervision. In *Proceedings of International Conference on Machine Learning*, pages 5583–5594. PMLR, 2021.
- [Kim, 2014] Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1746–1751, October 2014.
- [Kim et al., 2016] Yoon Kim, Yacine Jernite, David Sontag, and Alexander M Rush. Character-aware neural language models. In *Proceedings of the Thirtieth AAAI conference on artificial intelligence*, 2016.
- [Kiros et al., 2015] Ryan Kiros, Yukun Zhu, Russ R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Skip-thought vectors. *Advances in neural information processing systems*, 28, 2015.
- [Koehn, 2010] Philipp Koehn. *Statistical Machine Translation*. Cambridge University Press, 2010.
- [Krizhevsky et al., 2017] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.
- [Kumar et al., 2016] Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In *International conference on machine learning*, pages 1378–1387, 2016.
- [Lafferty et al., 2001] John Lafferty, Andrew McCallum, and Fernando Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the 18th International Conference on Machine Learning 2001*, pages 282–289, 2001.
- [Lai et al., 2015] Siwei Lai, Liheng Xu, Kang Liu, and Jun Zhao. Recurrent convolutional neural networks for text classification. In *Twenty-ninth AAAI conference on artificial intelligence*, 2015.
- [Lample et al., 2016] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 260–270, 2016.
- [Lang et al., 1990] Kevin J Lang, Alex H Waibel, and Geoffrey E Hinton. A time-delay neural network architecture for isolated word recognition. *Neural networks*, 3(1):23–43, 1990.
- [LeCun and Bengio, 1995] Yann LeCun and Yoshua Bengio. Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*, 3361(10):1995, 1995.
- [Li et al., 2020] Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li. A survey on deep learning for named entity recognition. *IEEE Transactions on Knowledge and Data Engineering*, 34(1):50–70, 2020.
- [Li et al., 2021] Junnan Li, Ramprasaath Selvaraju, Akhilesh Gotmare, Shafiq Joty, Caiming Xiong, and Steven Chu Hong Hoi. Align before fuse: Vision and language representation learning with momentum distillation. *Advances in neural information processing systems*, 34:9694–9705, 2021a.
- [Li et al., 2021] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of

- convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems*, 2021b.
- [Likhomanenko et al., 2021] Tatiana Likhomanenko, Qiantong Xu, Gabriel Synnaeve, Ronan Collobert, and Alex Rogozhnikov. Cape: Encoding relative positions with continuous augmented positional embeddings. *Advances in Neural Information Processing Systems*, 34:16079–16092, 2021.
- [Ling et al., 2015] Wang Ling, Chris Dyer, Alan W Black, Isabel Trancoso, Ramón Fernandez, Silvio Amir, Luis Marujo, and Tiago Luís. Finding function in form: Compositional character models for open vocabulary word representation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1520–1530, 2015.
- [Lippmann, 1989] Richard P Lippmann. Review of neural networks for speech recognition. *Neural computation*, 1(1):1–38, 1989.
- [Lipton et al., 2015] Zachary C Lipton, John Berkowitz, and Charles Elkan. A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019*, 2015.
- [Lopez, 2008] Adam Lopez. Statistical machine translation. *ACM Computing Surveys (CSUR)*, 40(3): 1–49, 2008.
- [Ma and Hovy, 2016] Xuezhe Ma and Eduard Hovy. End-to-end sequence labeling via bi-directional lstm-cnns-crf. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1064–1074, 2016.
- [Manning and Schütze, 1999] Chris Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. The MIT Press, 1999.
- [Manning et al., 2008] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [Markov, 1913] AA Markov. Essai d’une recherche statistique sur le texte du roman. *Eugene Onegin” illustrant la liaison des epreuve en chain* (‘Example of a statistical investigation of the text of “Eugene Onegin” illustrating the dependence between samples in chain”). In: *Izvestia Imperatorskoi Akademii Nauk (Bulletin de l’Académie Impériale des Sciences de St.-Petersbourg)*. 6th ser, 7:153–162, 1913.
- [Mikolov et al., 2010] Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Proceedings of Interspeech*, pages 1045–1048, 2010.
- [Mikolov et al., 2013] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR 2013)*, 2013.
- [Miller et al., 2016] Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston. Key-value memory networks for directly reading documents. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1400–1409, 2016.
- [Mitchell, 1997] Tom M. Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [Oord et al., 2018] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- [Pascanu et al., 2013] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. PMLR, 2013.

- [Pennington et al., 2014] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [Peters et al., 2018] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018.
- [Picone, 1993] Joseph W Picone. Signal modeling techniques in speech recognition. *Proceedings of the IEEE*, 81(9):1215–1247, 1993.
- [Rabiner and Juang, 1993] Lawrence Rabiner and Bing-Hwang Juang. *Fundamentals of speech recognition*. Prentice-Hall, Inc., 1993.
- [Reddy, 1976] D Raj Reddy. Speech recognition by machine: A review. *Proceedings of the IEEE*, 64(4):501–531, 1976.
- [Ross, 1924] William David Ross. *Aristotle’s metaphysics*. Clarendon Press, 1924.
- [Santos and Gatti, 2014] Cícero dos Santos and Maíra Gatti. Deep convolutional neural networks for sentiment analysis of short texts. In *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, pages 69–78, 2014.
- [Schneider et al., 2019] Steffen Schneider, Alexei Baevski, Ronan Collobert, and Michael Auli. wav2vec: Unsupervised pre-training for speech recognition. In *INTERSPEECH*, 2019.
- [Shannon, 1948] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.
- [Shaw et al., 2018] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, 2018.
- [Su et al., 2021] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [Sukhbaatar et al., 2015] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015.
- [Sundermeyer et al., 2012] Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. Lstm neural networks for language modeling. In *Proceedings of the Thirteenth annual conference of the international speech communication association*, 2012.
- [Sutskever, 2013] Ilya Sutskever. *Training recurrent neural networks*. University of Toronto Toronto, 2013.
- [Sutton and McCallum, 2012] Charles Sutton and Andrew McCallum. An introduction to conditional random fields. *Foundations and Trends® in Machine Learning*, 4(4):267–373, 2012.
- [Tai et al., 2015] Kai Sheng Tai, Richard Socher, and Christopher D Manning. Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1556–1566, 2015.
- [Tang et al., 2015] Duyu Tang, Bing Qin, and Ting Liu. Document modeling with gated recurrent

- neural network for sentiment classification. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1422–1432, 2015.
- [Tank and Hopfield, 1987] David W Tank and JJ Hopfield. Neural computation by concentrating information in time. *Proceedings of the National Academy of Sciences*, 84(7):1896–1900, 1987.
- [Trentin and Gori, 2001] Edmondo Trentin and Marco Gori. A survey of hybrid ann/hmm models for automatic speech recognition. *Neurocomputing*, 37(1-4):91–126, 2001.
- [Vaswani et al., 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Advances in Neural Information Processing Systems*, volume 30, 2017.
- [Vinyals et al., 2015] Oriol Vinyals, Łukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. *Advances in neural information processing systems*, 28, 2015.
- [Waibel et al., 1989] Alex Waibel, Toshiyuki Hanazawa, Geoffrey Hinton, Kiyohiro Shikano, and Kevin J Lang. Phoneme recognition using time-delay neural networks. *IEEE transactions on acoustics, speech, and signal processing*, 37(3):328–339, 1989.
- [Werbos, 1990] Paul J Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.
- [Weston et al., 2015] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *Proceedings of the 3rd International Conference on Learning Representations, ICLR 2015*, 2015.
- [Williams and Peng, 1990] Ronald J Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural computation*, 2(4):490–501, 1990.
- [Wu et al., 2016] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [Yang et al., 2016] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 1480–1489, 2016.
- [Zhang et al., 2021] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Dive into deep learning. *arXiv preprint arXiv:2106.11342*, 2021.
- [Zhang et al., 2015] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28, 2015.